

석 사 학 위 논 문

NVMe-oF 기반 Near-Data Processing을
통한 YOLO 동영상 전처리 파이프라인
최적화

김 기 태

부산대학교 대학원
정보융합공학과

2026년 8월

NVMe-of 기법 Near-Data Processing에 필요한 YOLO 네트웍을 전처리된 파이데이터로
추진하기

김기태

2029년
8월

NVMe-oF 기반 Near-Data Processing을 통한 YOLO 동영상 전처리 파이프라인 최적화

이 논문을 공학석사 학위논문으로 제출함

김 기 태

부산대학교 대학원

정보융합공학과

지 도 교 수 안 성 용

김기태의 공학석사 학위논문을 인준함

2026년 6월 29일

위원장	권 동 현	인
-----	-------	---

위 원	손 준 영	인
-----	-------	---

위 원	안 성 용	인
-----	-------	---

차 례

<제목 차례>

I. 서론	1
1. 연구의 배경과 목적	1
가. 연구의 배경	1
나. 연구의 목적	2
II. 배경	3
1. 분리형 스토리지 아키텍처	3
가. 배경 및 개념	3
나. NVMe-oF 개요 및 TCP 기반 동작 방식	4
2. SPDK(Storage Performance Development Kit)	6
가. 커널 바이패스 및 사용자 공간 드라이버 개념	6
나. NVMe-oF Target으로서의 동작 구조	7
3. Near-Data Processing (NDP)	8
가. 개념 및 등장 배경	8
4. 영상 객체 탐지 파이프라인	9
가. DNN 기반 객체 탐지	9
나. YOLO 추론 구조 및 입력 전처리	10
다. 추론 파이프라인 및 탐지 성능 평가	11
III. 관련 연구	13
1. 소프트웨어 정의 NDP 플랫폼 연구	13
2. 영상 분석 파이프라인의 연산 오프로딩 연구	14
IV. 설계 및 구현	15
1. 시스템 아키텍처	15
가. 전체 시스템 구성	15
2. NDP 명령어 인터페이스 설계	18
가. 0xC0 NDP 처리 트리거 명령어	18
나. 0xC2 결과 회수 명령어	19
3. 스토리지 노드 구현	20
4. 호스트 노드 구현	21

V. 실험 결과 및 분석	23
1. 실험 환경	23
가. 하드웨어 구성	23
나. 소프트웨어 스택	23
다. 실험 데이터셋	24
2. 비교 기준선 정의	25
가. 기존 호스트 중심 처리 파이프라인	25
나. NDP 오프로딩 파이프라인	25
다. 측정 방법 및 지표 정의	26
3. 파이프라인 처리 지연 분석	27
가. 종단 간 지연 시간	27
나. 지연 시간 세부 분석	28
4. 데이터 전송량 및 전송 시간 비교	29
5. 메모리 사용량 분석	32
VI. 결론 및 향후 연구	33
1. 결론	33
2. 향후 연구	33
가. CXL 기반 메모리 시맨틱 NDP 아키텍처로의 확장	33
나. 다중 스토리지 노드 분산 처리 아키텍처	34
다. 전용 연산 가속기 탑재를 통한 전처리 성능 향상	34
라. 적응형 프레임 샘플링 및 전처리 정책	35

<표 차례>

[표1] 실험 환경 구성	23
[표2] 소프트웨어 환경 구성	24
[표3] 입력 데이터 크기	25
[표4] 성능 비교를 위한 측정 항목 정의	26
[표5] 호스트 및 시스템 전체 메모리 사용량 비교	32

<그림 차례>

[그림 1] 통합형 아키텍처(좌)와 분리형 스토리지 아키텍처(우) 구조 비교	3
[그림 2] NVMe/TCP 기반 호스트-타겟 간 통신 동작 시퀀스	4
[그림 3] NVMe/TCP PDU 구조	5
[그림 4] 기존 Linux I/O 경로(좌)와 SPDK 커널 바이패스 경로(우) 비교	6
[그림 5] SPDK NVMe-oF Target의 계층적 동작 구조	7
[그림 6] 일반적인 방식(좌)과 NDP 방식(우) 개념 비교	8
[그림 7] DNN 기반 영상 객체 탐지 파이프라인	10
[그림 8] IoU 기반 탐지 평가 과정	11
[그림 9] Near Data Processing 플랫폼 아키텍처 [4]	13
[그림 10] ISVDA 아키텍처 [5]	14
[그림 11] 제안하는 NDP 플랫폼의 시스템 아키텍처	15
[그림 12] NDP 시스템 YOLO 영상 객체 탐지 전처리 파이프라인	17
[그림 13] 커스텀 NVMe-oF 명령어 기반 NDP 전처리 및 결과 회수 시퀀스 다이어그램 18	
[그림 14] 종단 간 지연 시간	27
[그림 15] 파이프라인 단계별 지연 시간 분해	28
[그림 16] 스토리지-호스트 간 데이터 전송량 비교	30
[그림 17] 스토리지-호스트 간 데이터 전송시간 비교	31
[그림 18] 스토리지-호스트 간 전체 실행시간 대비 데이터 전송시간 비율 비교	31

NVMe-oF 기반 Near-Data Processing을 통한 YOLO 동영상 전처리 파이프라인 최적화

김 기 태

부산대학교 대학원 정보융합공학과

요약

최근 스마트 시티 및 보안 인프라 확산으로 CCTV등 다양한 영상정보 기반의 객체 탐지 수요가 급증하면서 YOLO와 같은 DNN 기반 객체 탐지 모델이 널리 활용되고 있다. 그러나 객체 탐지를 위한 동영상 전처리 단계에서 대용량 동영상 데이터를 스토리지에서 호스트로 전송하기 위해 발생하는 입출력 대역폭 과부하와 과도한 메모리 사용이 성능 병목의 원인이 되고 있다. 본 논문에 서는 이를 해결하기 위해 Near-Data Processing (NDP) 기반의 비디오 디코딩 및 데이터 전처리 아키텍처를 제안한다. 제안된 기법은 스토리지 노드에서 비디오 디코딩, 프레임 샘플링 및 YOLO 입력 형식에 맞는 전처리를 수행한 후 결과만을 호스트로 전달한다. 이를 위해 NVMe over Fabrics (NVMe-oF)을 확장해 커스텀 커맨드를 이용한 동영상 전처리 연산 오프로딩 프로토콜을 구현하였으며 이를 기존 YOLOv7-tiny 기반 Darknet 추론 파이프라인에 통합해 성능 평가를 진행 하였다. 실험 결과, 제안 기법은 기존 방식 대비 Storage-Host 데이터 전송량을 최대 92% 감소시키고 End-to-End 지연시간을 최대 13.9배 단축하였다. 이를 통해 분리형 스토리지 환경에서 객체 탐지 파이프라인의 효율을 크게 향상시킬 수 있음을 확인하였다.

I. 서론

1. 연구의 배경과 목적

가. 연구의 배경

최근 스마트시티 및 보안 인프라의 급속한 확대로 인해 고해상도 CCTV를 비롯한 영상정보 기반의 객체 탐지 기술이 산업 전반에서 주목받고 있다. 특히 Deep Neural Network(DNN) 기반 객체 탐지 기술의 추론 성능이 비약적으로 향상되면서, 보안 관제·교통 감시·재난 예방 등 공공 안전 분야의 핵심 기술로 자리 잡고 있다.

그러나 객체 탐지 모델을 실제 서비스 환경에 적용하기 위해서는 대용량 동영상 데이터에 대한 전처리 파이프라인이 선행되어야 한다. 이 과정에서 스토리지로부터 데이터를 읽어 메모리로 이동시키는 데이터 전송 비용이 현대의 데이터 중심 컴퓨팅 환경에서 병목으로 작용하고 있다. 이러한 문제는 컴퓨팅 자원과 스토리지 자원을 물리적으로 분리하여 유연한 자원 확장을 가능하게 하는 분리형 스토리지(Disaggregated Storage) 아키텍처 환경에서 더욱 심화된다. 분리형 환경에서는 스토리지 노드와 컴퓨팅 노드 사이의 모든 데이터 이동이 네트워크 패브릭을 경유해야 하므로, 전송 지연 및 대역폭 포화에 따른 병목 현상이 구조적으로 내재되어 있다.

이러한 병목을 근본적으로 해소하기 위해서는 데이터를 호스트로 이동시키기 전에 스토리지 계층에서 직접 처리하는 Near-Data Processing(NDP) 접근 방식이 필요하다. 기존의 Computational Storage Device(CSD)를 활용한 NDP 구현은 이러한 개념을 실현하는 대표적인 방식이나, 특수 목적의 하드웨어나 커널 수정이 수반되어 실제 데이터 센터 환경에 배포하기에 현실적인 제약이 따른다.

반면, NVMe-oF(NVMe over Fabrics) TCP 기반의 분리형 스토리지 구조는 별도의 하드웨어 수정 없이 기존 이더넷 인프라를 그대로 활용하므로, 소프트웨어 정의 방식으로 NDP를 구현할 수 있는 실용적인 대안이다. 여기에 SPDK(Storage Performance Development Kit)와 같은 사용자 공간(user space) 기반의 고성능 스토리지 드라이버를 활용하면, 커널 개입 없이 NVMe SSD에 직접 접근하여 스토리지 노드에서 데이터 처리 연산을 직접 수행할 수 있다. 나아가 최신 처리 장치에 내장된 하드웨어 비디오 가속기(H.264, H.265, AV1 등)를 연계하면, 호스트 CPU의 개입 없이 스토리지 노드 측에서 고속 디코딩 및 필터링을 수행하는 것이 가능할 것으로 기대된다.

특히 YOLO(You Only Look Once)[13]와 같은 실시간 객체 탐지 모델은 추론 연산 속도가 크게 향상되었음에도 불구하고, 정작 모델에 입력될 데이터를 준비하는 전처리 파이프라인이 전체 시스템 성능을 제약하는 병목으로 부각되고 있다.[14] 고해상도 영상의 모든 영역과 프레임이 분석에 기여하는 것이 아님에도, 기존 호스트 기반 전처리 방식은 필터링 이전의 원본 데이터를 그대로 네트워크를 통해 전송해야 한다. 이는 분석에 불필요한 프레임을 선별하는 샘플링이 수행되기 이전에 이미 대역폭 낭비가 발생함을 의미하며, 구조적으로 비효율적인 데이터 이동이 지속될 수밖에 없다.

따라서 지속 가능한 고성능 영상 분석 환경을 구축하기 위해서는, 데이터의 의미를 선제적으로 파악하고 유효한 정보만을 선별하여 전송하는 NDP 기반의 스토리지 사이드 전처리 오프로딩 접근이 필요하다.

나. 연구의 목적

본 논문은 분리형 스토리지 환경에서 CCTV 영상 분석 워크로드를 대상으로, 추가적인 하드웨어 수정 없이 SPDK 기반의 사용자 레벨 드라이버 확장을 통해 비디오 전처리를 스토리지 노드에서 직접 수행하는 소프트웨어 기반 NDP 최적화 아키텍처를 제안하고 그 실효성을 검증하는 것을 목적으로 한다.

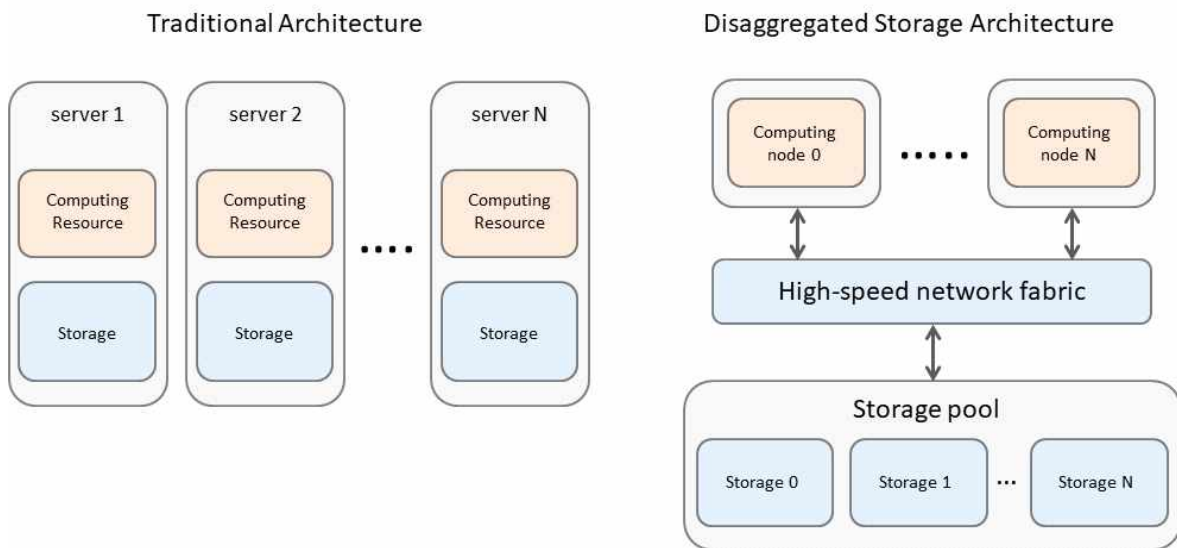
이를 위해 본 논문에서는 SPDK 기반 스토리지 노드에서 비디오 디코딩, 프레임 샘플링을 포함한 전처리 파이프라인을 수행하고 전처리된 결과만을 컴퓨팅 노드로 전달하는 소프트웨어 아키텍처를 설계 및 구현한다. 제안 아키텍처는 데이터 전송량을 최소화하는 방향으로 설계된다. 이렇게 구현된 제안 시스템은 SPDK 미적용 NVMe-oF 환경에서 원본 영상 데이터를 호스트로 전송 및 전처리한 후 Darknet YOLO 추론을 수행하는 기존 방식을 Baseline으로 설정하여 정량적으로 비교·평가하며, 데이터 전송량, 호스트 및 스토리지 노드 양측의 메모리 사용량, 종단 간 지연시간을 주요 평가 지표로 삼는다.

이를 통해 소프트웨어 정의 방식의 NDP가 분리형 스토리지 환경에서 실질적인 성능 개선 수단으로 기능할 수 있음을 보이고, 데이터 전송 병목 완화와 호스트 자원 효율화를 동시에 달성하는 영상 분석 파이프라인을 최적화할 수 있음을 보이고자 한다

II. 배경

1. 분리형 스토리지 아키텍처

가. 배경 및 개념



[그림 1] 통합형 아키텍처(좌)와 분리형 스토리지 아키텍처(우) 구조 비교

전통적인 서버 환경에서는 컴퓨팅 자원과 스토리지 자원이 하나의 물리적 노드 안에 함께 구성되는 통합형 아키텍처가 일반적이었다. 이 구조는 로컬 PCIe 버스를 통한 낮은 접근 지연 시간이라는 장점이 있으나, 컴퓨팅 자원과 스토리지 자원의 수요가 항상 비례하지 않아 자원 확장 측면에서 불균형이 발생하기도 한다. 대용량 영상 데이터 처리와 같이 스토리지 집약적인 워크로드에서는 스토리지 용량 확장을 위해 불필요한 컴퓨팅 자원까지 함께 증설해야 하는 낭비가 발생하며, 서버 간 스토리지 데이터를 공유하려면 별도의 상위 네트워크 계층을 거쳐야 하는 비효율도 따른다.

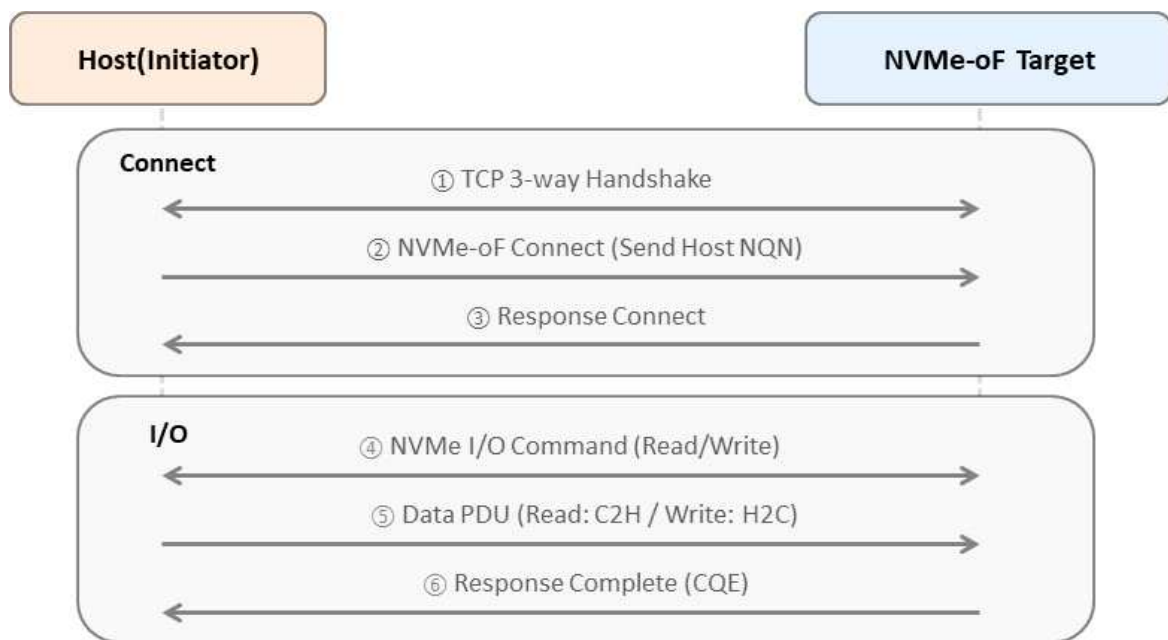
이러한 문제를 해결하기 위해 등장한 것이 분리형 스토리지 아키텍처(Disaggregated Storage Architecture)[10]이다. 이 구조는 컴퓨팅 노드와 스토리지 노드를 물리적으로 분리하고 고속 네트워크 패브릭(fabric)으로 연결함으로써, 스토리지 풀을 여러 컴퓨팅 노드가 공유하고 각 자원을 독립적으로 확장(scale-out)할 수 있게 한다.

분리형 스토리지 아키텍처가 실용화될 수 있었던 핵심 배경에는 두 가지 기술적 발전이 있다. 첫째, NVMe(Non-Volatile Memory Express) SSD의 보급으로 스토리지 장치 자체의 접근 지연 시간이 수십 마이크로초 수준으로 단축되었다. 기존 SATA 기반 HDD

는 수 밀리초 단위의 지연이 발생해 네트워크 경우 자체가 무의미했으나, NVMe SSD는 지연 시간이 충분히 짧아 네트워크 오버헤드를 더하더라도 전체적으로 합리적인 성능을 낼 수 있다. 둘째, 100Gbps급 고속 이더넷의 데이터센터 확산으로 네트워크 전송 대역폭과 지연 시간이 크게 개선되었다. 이로 인해 네트워크를 통한 스토리지 접근이 로컬 PCIe 접근과의 성능 격차를 현실적으로 좁히는 수준에 이르렀다.

결과적으로 분리형 스토리지 아키텍처는 대규모 클라우드 환경 및 데이터센터에서 자원 활용 효율을 높이는 핵심 인프라 패러다임으로 자리잡고 있으며, 이를 효율적으로 지원하는 프로토콜 표준이 NVMe-oF이다.

나. NVMe-oF 개요 및 TCP 기반 동작 방식



[그림 2] NVMe/TCP 기반 호스트-타겟 간 통신 동작 시퀀스

NVMe over Fabrics(NVMe-oF)[6]는 NVMe 명령 집합을 네트워크 패브릭 위에서 동작할 수 있도록 확장한 표준 규격으로, NVM Express 조직이 2016년에 표준화하였다. NVMe-oF는 호스트(initiator)가 원격 스토리지 장치를 구성하는 타겟(target)에 NVMe 명령을 전송하고 응답을 수신하는 클라이언트-서버 모델을 따른다. 이를 통해 호스트는 물리적으로 분리된 NVMe 장치를 마치 로컬 PCIe 장치처럼 접근할 수 있으며, NVMe 명령 집합을 변경 없이 그대로 사용할 수 있다.

NVMe-oF는 전송 계층(transport)을 추상화하는 구조로 설계되어, RoCE(RDMA

over Converged Ethernet)나 InfiniBand와 같은 RDMA 기반 전송도 지원한다. 그러나 RDMA 방식은 전용 하드웨어(RNIC)가 필요해 인프라 구축 비용이 높다. 이에 반해 NVMe/TCP[7]는 기존 TCP/IP 네트워크 인프라와 범용 이더넷 NIC만으로 동작하는 방식으로, 별도의 전용 하드웨어 없이 구성 가능하다는 범용성 때문에 최근 데이터센터 환경에서 널리 채택되고 있다[11].

NVMe/TCP의 동작 방식은 다음과 같다. 호스트는 타겟의 IP 주소와 TCP 포트(기본 값: 4420)로 TCP 연결을 수립한 뒤, NVMe-oF Fabric Connect 명령을 통해 서브시스템(subsystem)과의 논리적 연결을 확립한다. 이 과정에서 호스트와 타겟은 NQN(NVMe Qualified Name)으로 서로를 식별한다. 연결 수립 후에는 I/O 큐 페어(Submission Queue / Completion Queue)를 생성하고, 이를 통해 Read/Write 명령을 TCP 스트림 위에 캡슐화하여 전송한다.



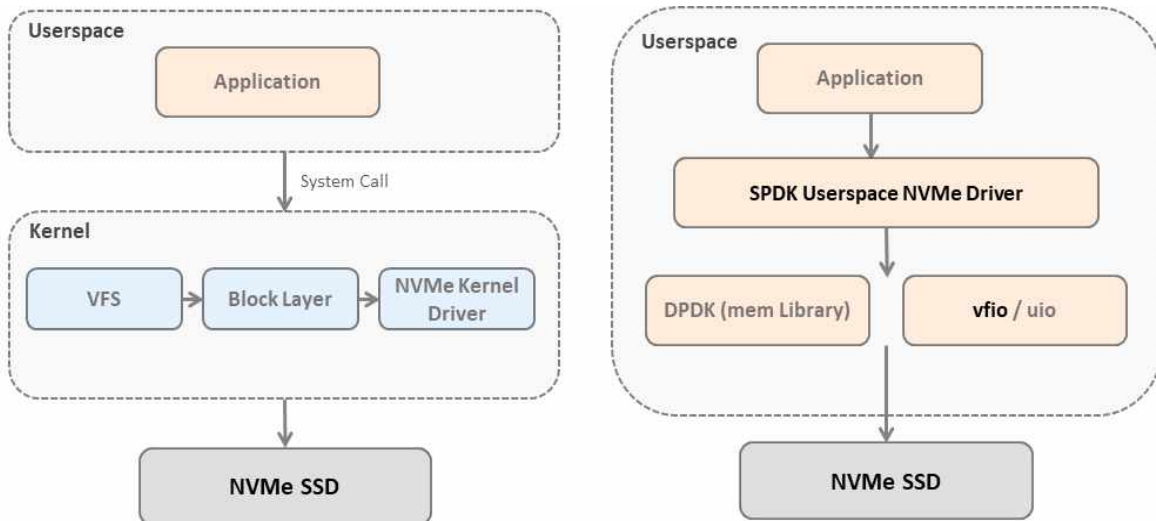
[그림 3] NVMe/TCP PDU 구조

NVMe/TCP의 PDU(Protocol Data Unit)는 공통 헤더(Common Header), NVMe 명령 캡슐(Command Capsule), 그리고 데이터 페이로드로 구성된다. 데이터 전송 방향에 따라 타겟에서 호스트 방향으로 읽기 데이터를 전송하는 C2H(Controller-to-Host) PDU와, 쓰기 데이터 요청에 사용되는 H2C(Host-to-Controller) PDU로 구분된다. 또한 PDU에는 헤더 다이제스트(header digest)와 데이터 다이제스트(data digest)를 선택적으로 활성화하는 무결성 보장 메커니즘이 포함된다.

NVMe/TCP는 RDMA 기반 방식에 비해 지연 시간이 다소 높을 수 있으나, 범용 이더넷 NIC만으로 동작한다는 점에서 인프라 구축 비용 측면에서 유리하다. 특히 본 연구에서 활용하는 SPDK(Storage Performance Development Kit)와 같이 커널 바이패스 기반의 고성능 사용자 공간 스택을 타겟 측에 결합하면, 소프트웨어 오버헤드를 최소화하여 NVMe/TCP 환경에서도 수십 마이크로초 수준의 왕복 지연 시간(round-trip latency)을 달성할 수 있다. 이는 CCTV 영상 전처리 및 객체 탐지 워크로드를 스토리지 계층에서 수행하는 본 연구의 NDP 구조와 직접적으로 연결된다.

2. SPDK(Storage Performance Development Kit)

가. 커널 바이패스 및 사용자 공간 드라이버 개념



[그림 4] 기존 Linux I/O 경로(좌)와 SPDK 커널 바이패스 경로(우) 비교

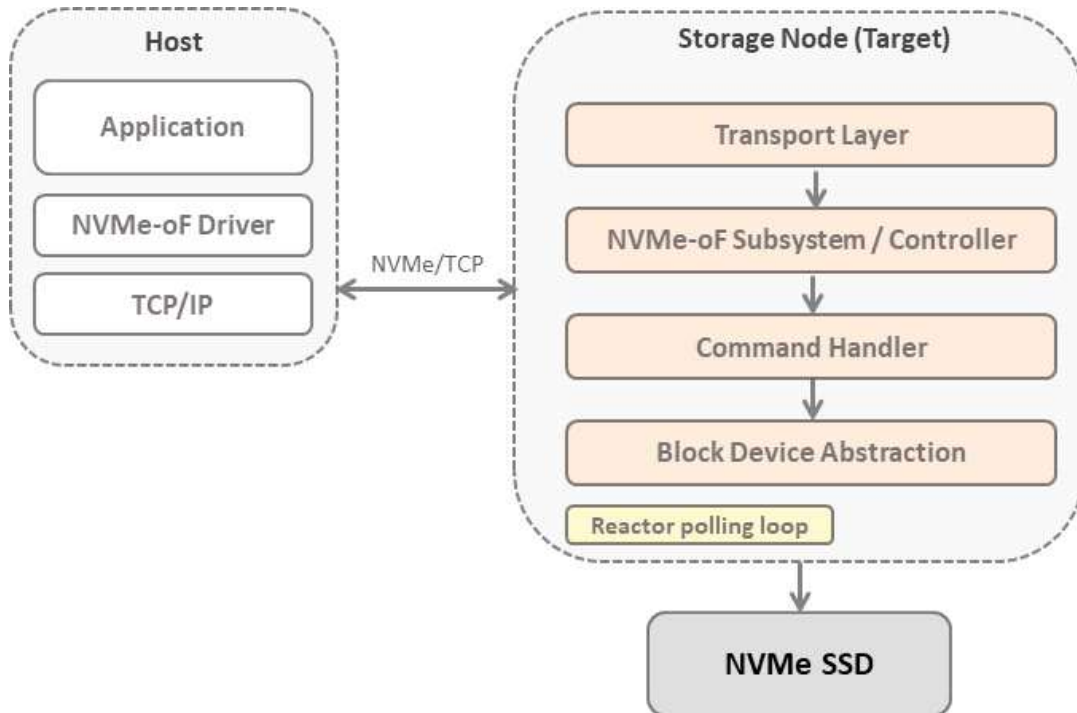
일반적인 Linux 환경에서 애플리케이션이 스토리지 장치에 접근하려면 시스템 콜(system call)을 통해 커널에 제어권을 넘긴 뒤, VFS(Virtual File System), 블록 계층(block layer), 그리고 커널 디바이스 드라이버를 순차적으로 통과해야 한다. 이 과정에서는 사용자 공간과 커널 공간 사이의 컨텍스트 전환(context switch)이 반복적으로 발생하며, 인터럽트 처리와 다단계 소프트웨어 스택을 경유하는 데 따른 지연도 누적된다. 하드웨어 접근 지연이 수 밀리초 수준이었던 HDD 시대에는 이러한 소프트웨어 오버헤드가 상대적으로 무시할 만했으나, NVMe SSD의 하드웨어 접근 지연이 수 마이크로초 수준으로 단축된 현재에는 커널 스택 경유에 따른 소프트웨어 오버헤드가 전체 I/O 지연 시간에서 차지하는 비중이 점차 커지게 되었다.

SPDK[12]는 이러한 소프트웨어 스택 오버헤드를 근본적으로 해소하기 위해 등장한 고성능 스토리지 소프트웨어 개발 키트이다. SPDK는 NVMe 장치를 커널 드라이버(nvme.ko) 대신 uio(Userspace I/O) 또는 vfio(Virtual Function I/O) 인터페이스를 통해 사용자 공간 프로세스에 직접 매핑하고, 사용자 공간에서 동작하는 완전한 NVMe 드라이버로 장치를 제어한다. 그 결과 I/O 경로 전체가 커널을 제외한 사용자 공간 내에서 이루어지며, 커널 개입에 따른 컨텍스트 전환과 인터럽트 처리 오버헤드가 구조적으로 제거된다.

커널을 우회한 구조에서는 I/O 완료를 인터럽트로 통보받을 수 없으므로, SPDK는

전용 CPU 코어에 배정된 리액터(reactor) 스레드가 NVMe CQ(Completion Queue)를 지속적으로 순환 검사하는 폴링(busy-polling) 방식으로 완료를 감지한다. 이와 더불어 SPDK는 DPDK의 Hugepage 메모리 풀을 활용하여 물리 주소가 고정된(pinned) DMA 전용 메모리를 제공함으로써, NVMe 장치가 일관된 물리 주소로 직접 데이터를 읽고 쓸 수 있도록 보장한다.

나. NVMe-oF Target으로서의 동작 구조



[그림 5] SPDK NVMe-oF Target의 계층적 동작 구조

SPDK는 스토리지 드라이버 라이브러리 역할뿐만 아니라, 서버 머신을 NVMe-oF 타겟으로 동작시킬 수 있는 NVMe-oF Target 프레임워크를 함께 제공한다. 이 프레임워크는 전송 계층, 서브시스템·컨트롤러 계층, 명령 처리 계층, bdev 추상화 계층의 네 단계로 구성된다.

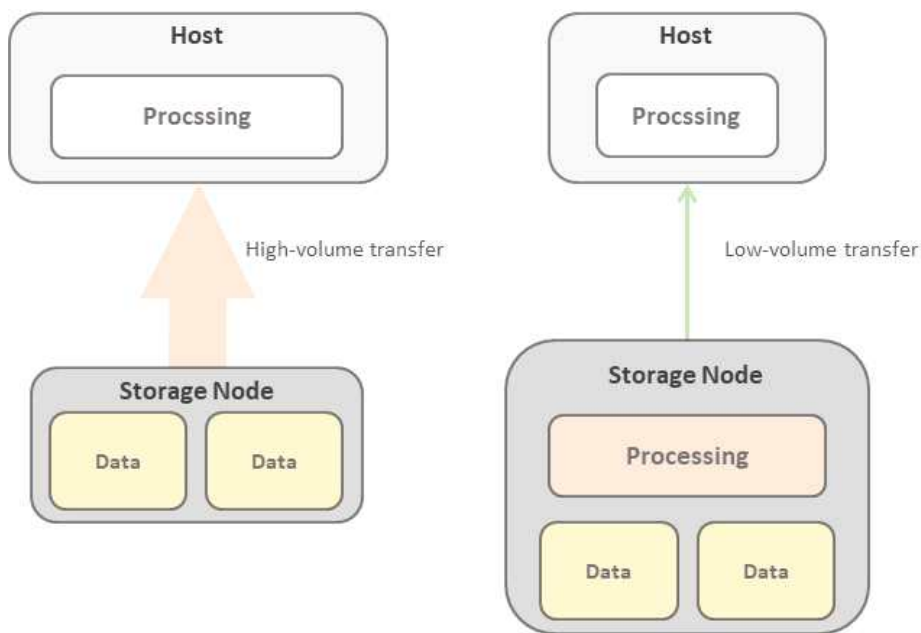
전송 계층은 NVMe/TCP 큐 페어를 통해 수신된 PDU를 파싱하고 NVMe 명령 캡슐을 추출하여 상위 계층으로 전달한다. 서브시스템·컨트롤러 계층에서는 NQN 기반 인증, 큐 쌍 할당, 네임스페이스 접근 제어가 이루어지며, 명령의 종류에 따라 적절한 처리 경로로 분기한다. 명령 처리 계층에서는 표준 NVMe I/O 명령을 처리하는 경로 외에, 벤더 특화 명령어(Vendor Specific Command)의 opcode에 대해 사용자 정의 핸들러를

등록하는 인터페이스를 제공한다. 이를 통해 타겟의 동작을 단순한 데이터 서빙에 더하여 NDP와 같은 응용으로 유연하게 확장할 수 있다.

가장 하위의 bdev 추상화 계층은 실제 스토리지 장치의 종류와 무관하게 일관된 비동기 I/O 인터페이스를 제공한다. 명령 처리 계층이 bdev API를 통해 I/O를 요청하면 즉시 반환하며, 리액터 폴링 루프가 NVMe CQ를 감시하다가 완료 이벤트 발생 시 등록된 콜백을 실행하여 응답을 생성한다. 이처럼 커널 바이패스, 폴링 기반 완료 감지, 비동기 처리, 그리고 확장 가능한 명령 처리 구조가 결합된 SPDK는 고성능 분리형 스토리지 인프라를 구축하는 데 적합한 플랫폼으로 활용된다.

3. Near-Data Processing (NDP)

가. 개념 및 등장 배경



[그림 6] 일반적인 방식(좌)과 NDP 방식(우) 개념 비교

전통적인 컴퓨팅 모델에서는 데이터를 저장 장치에서 읽어 네트워크 또는 버스를 통해 컴퓨팅 자원으로 옮긴 뒤 처리하는 연산 중심 구조가 표준이었다. 이 구조는 연산 능력이 희소하고 데이터 규모가 작았던 시대에는 합리적이었으나, 빅데이터·영상 스트림·센서 로그 등 처리해야 할 데이터의 규모가 폭발적으로 증가하면서 데이터 이동 비용이 전체 처리 성능에서 차지하는 비중이 점차 커지게 되었다. 데이터를 이동시키는 데 소요

되는 네트워크 대역폭 비용과 지연이 연산 비용보다 오히려 커지는 상황이 빈번해진 것이다. 이러한 현상을 데이터 중력(Data Gravity)이라 부르며, 데이터를 연산 쪽으로 이동시키는 대신 연산을 데이터 가까이로 옮기는 패러다임의 전환이 요구되었다.

이러한 발상은 이미 1990년대 후반부터 탐구되었다. Acharya et al.은 1998년 활성 디스크(Active Disk)[1] 개념을 제안하며, 스토리지의 남은 자원을 활용해 장치 내부에서 직접 연산을 수행함으로써 호스트로 전송해야 하는 데이터 양을 줄일 수 있음을 보였다. 이 개념은 이후 스토리지 장치의 프로그래밍 유연성이 높아지면서 Willow [2], Biscuit [3] 등의 연구로 이어졌으며, 이를 포괄하는 용어로 Near-Data Processing(NDP)이 사용되기 시작하였다.

NDP의 핵심 원칙은 단순하다. 데이터를 필요한 만큼 연산 노드로 전송하는 대신, 데이터가 위치한 곳에서 1차 처리를 수행하고 그 결과만을 전달한다. 원시 데이터 전체를 이동시킬 필요가 없으므로 네트워크 대역폭 소비가 줄고, 불필요한 데이터 전송에 따른 지연도 함께 감소한다.

분리형 스토리지 환경에서는 스토리지 노드가 NVMe-oF 타겟을 운용하기 위한 범용 CPU를 보유한다. 이 CPU는 이미 스토리지 장치에 인접한 위치에 존재하므로, 데이터를 컴퓨팅 노드로 전달하기 전에 1차 처리를 수행할 수 있는 자원으로 활용될 수 있다. 이는 전용 하드웨어 없이도 소프트웨어 정의 NDP를 구성할 수 있는 자연스러운 기반이 된다.

4. 영상 객체 탐지 파이프라인

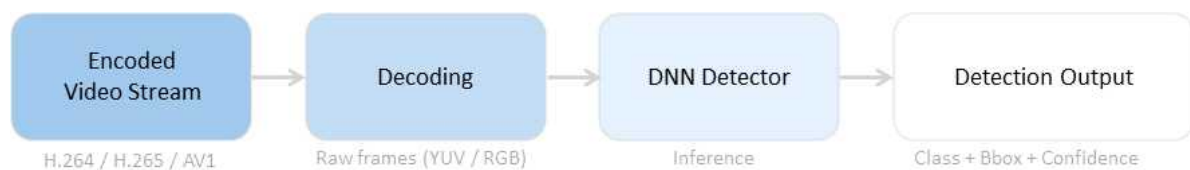
가. DNN 기반 객체 탐지

객체 탐지(Object Detection)는 이미지 내에서 특정 클래스의 객체가 존재하는 위치를 경계 상자(Bounding Box, Bbox)로 나타내고, 동시에 클래스를 분류하는 작업이다. 사람이 직접 정의한 규칙과 특징에 의존하는 전통적 방식은 조명 변화, 물체 가림, 복잡한 배경 등 다양한 환경 조건에서 탐지 정확도가 떨어지는 경향이 있었다. 반면 DNN(Deep Neural Network) 기반 접근은 대규모 데이터셋으로부터 계층적 특징을 자동으로 학습하여 다양한 환경에서 높은 탐지 정확도를 달성하였으며, 현재 실용적 객체 탐지의 표준적 방법론으로 자리잡고 있다.

동영상에서 객체를 탐지하기 위해서는 먼저 압축된 비디오 스트림을 개별 이미지 프

프레임으로 변환하는 과정이 필요하다. 영상 저장 및 전송에 널리 사용되는 H.264와 H.265(HEVC)는 시간적·공간적 중복성을 제거하는 손실 압축 방식으로, I-프레임(인트라 프레임), P-프레임(예측 프레임), B-프레임(양방향 예측 프레임)으로 구성된 GOP(Group of Pictures) 구조를 사용한다. 이 구조에서 특정 프레임을 복원하려면 선행 참조 프레임이 먼저 디코딩되어야 하므로, 소프트웨어 디코딩은 순차적 의존성을 가진 연산 과정이다. H.265는 H.264 대비 동일 화질에서 약 절반의 비트레이트를 달성하지만, 더 복잡한 예측 구조로 인해 디코딩 연산 부하가 더 높다.

디코딩을 통해 복원된 원시 프레임은 전처리를 거쳐 DNN 모델에 입력되고, 모델의 예측 결과로부터 경계 상자와 클래스 레이블이 출력된다.



[그림 7] DNN 기반 영상 객체 탐지 파이프라인

나. YOLO 추론 구조 및 입력 전처리

DNN 기반 객체 탐지 모델은 크게 두 단계(two-stage) 방식과 단일 단계(single-stage) 방식으로 나뉜다. 두 단계 방식은 객체가 있을 법한 영역을 먼저 제안한 뒤 각 영역을 분류하는 구조로 정확도가 높으나, 두 단계를 순차적으로 처리하는 특성상 실시간 워크로드에 적합하지 않다. YOLO(You Only Look Once)[8][13]는 하나의 신경망이 이미지를 한 번만 통과하여 위치와 클래스를 동시에 예측하는 단일 단계 방식으로, 추론 속도와 탐지 정확도 간의 균형 측면에서 실용적인 선택지로 널리 사용된다.

DNN 모델은 원시 프레임을 그대로 입력으로 받을 수 없으며, 고정된 크기와 값 범위의 전처리가 선행되어야 한다. 입력 크기를 $N \times N$ 픽셀로 통일하는 이유는 완전 연결 계층(fully-connected layer)의 가중치 행렬이 고정된 입력 차원을 전제로 구성되기 때문이다. 예를 들어 416×416 으로 학습된 모델에 640×480 프레임을 그대로 입력하면 가중치 차원이 맞지 않아 추론이 불가능하다. 입력 크기가 클수록 작은 객체 탐지에 유리하지만 연산 비용이 증가하므로, 해상도 선택은 정확도와 처리 속도 간의 균형을 고려하여 결정한다.

픽셀 값을 0~255 정수에서 0.0~1.0 실수 범위로 정규화하는 것은 수치적 안정성을

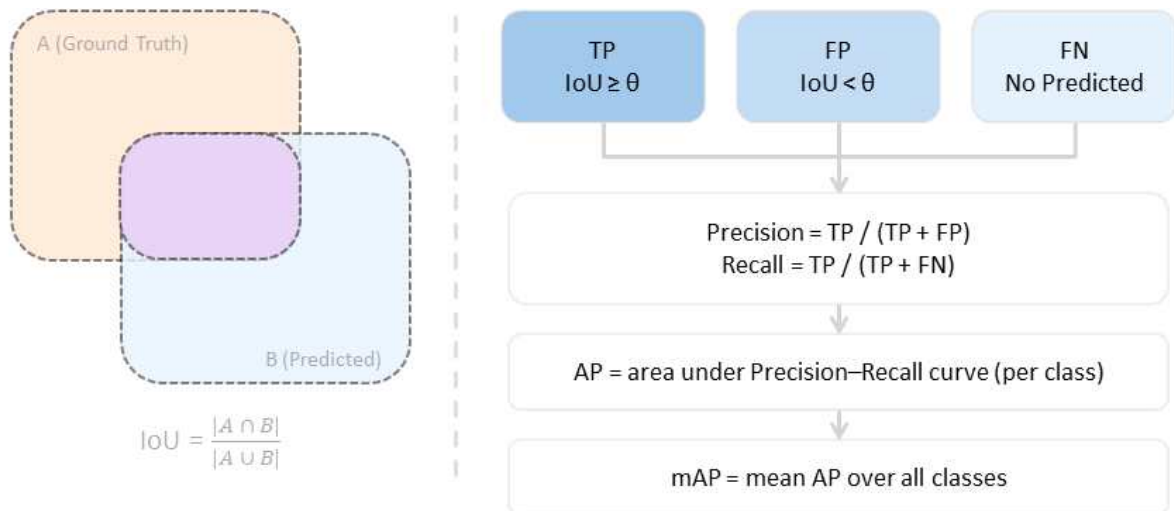
위한 표준적 처리이다. 정규화되지 않은 상태에서는 255에 가까운 값들이 계층을 거치며 활성화 값이 과도하게 커지는 현상이 발생할 수 있다. 또한 야간 영상과 주간 영상처럼 밝기 범위가 크게 다른 입력들도 정규화를 통해 동일한 0~1 범위로 맞춰짐으로써, 모델이 조명 조건에 관계없이 일관된 분포로 입력을 처리할 수 있게 된다.

YOLO의 네트워크는 입력 이미지 전체를 $N \times N$ 그리드로 분할하고, 각 셀이 미리 정의된 크기와 비율의 후보 상자인 앵커 박스를 기준으로 경계 상자의 위치와 크기를 예측한다. 이와 함께 각 셀은 해당 위치에 객체가 존재할 가능성을 나타내는 신뢰도(confidence)와 어떤 클래스에 속하는지를 나타내는 클래스 확률을 동시에 출력한다. 예를 들어 13×13 그리드에서 사람이 이미지 중앙에 있다면, 해당 위치를 담당하는 셀의 신뢰도는 높게, 주변 빈 공간을 담당하는 셀의 신뢰도는 낮게 출력되어 객체가 없는 셀은 이후 처리에서 자동으로 제외된다. 다중 스케일 헤드는 서로 다른 해상도의 특징 맵에서 예측을 병렬로 수행하므로, 크기가 다양한 객체를 하나의 순전파로 처리할 수 있다. 다만 인접한 여러 셀이 동일한 객체에 대해 중복된 상자를 예측하는 경우가 발생하는데, NMS(Non-Maximum Suppression)는 이를 제거하는 후처리 과정이다. 예를 들어 한 사람에 대해 신뢰도 0.95, 0.88, 0.72짜리 상자가 각각 예측되었다면, 가장 신뢰도가 높은 0.95 상자를 남기고 이와 많이 겹치는 나머지 두 상자를 제거하여 최종적으로 하나의 탐지 결과만을 출력한다.

다. 추론 파이프라인 및 탐지 성능 평가

Darknet[9]은 YOLO의 원저자인 Redmon이 C 언어로 개발한 경량 딥러닝 프레임워크로, YOLO 모델의 학습 및 추론에 특화되어 있다. 외부 의존성이 적어 이식성이 높고, CUDA 없이 CPU만으로도 추론을 수행할 수 있으며 CUDA 활성화 시 GPU 가속도 지원한다. 추론 파이프라인은 모델 구성 파일(.cfg)과 사전 학습 가중치(.weights)를 로드하여 네트워크를 초기화하고, 입력 이미지를 리사이즈·정규화한 뒤 순전파를 수행하여 예측 텐서를 얻는다. 이후 신뢰도 임계값 필터링과 NMS를 적용하여 클래스 레이블, 신뢰도 점수, 경계 상자 좌표로 구성된 최종 탐지 결과를 반환한다.

탐지 모델의 성능을 정량적으로 평가하기 위해서는 예측된 경계 상자가 실제 정답(ground truth)과 얼마나 일치하는지를 측정하는 기준이 필요하다. 이를 위해 IoU(Intersection over Union)가 사용되며, 예측 상자와 정답 상자의 교집합 넓이를 합집합 넓이로 나눈 값으로 정의된다. 예를 들어 두 상자가 절반 정도 겹친다면 IoU는 약



[그림 8] IoU 기반 탐지 평가 과정

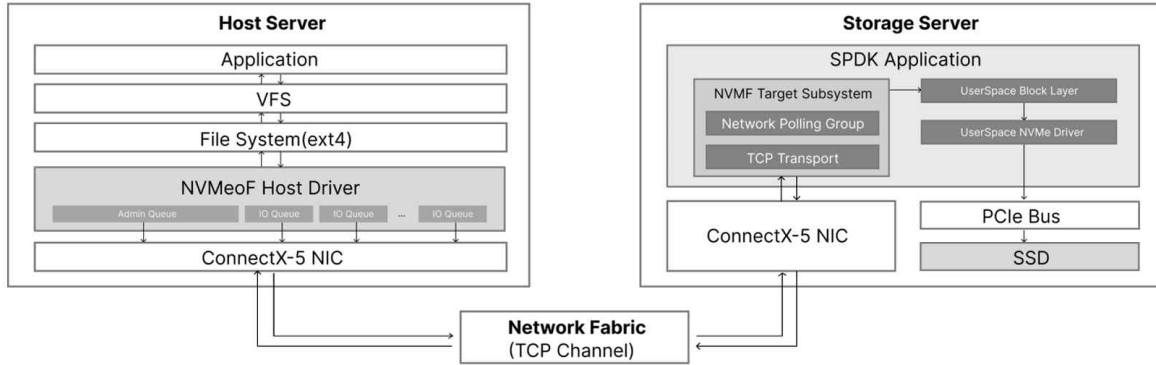
0.33이며, 완전히 일치하면 1.0이 된다.

사전에 정의한 IoU 임계값 θ 를 기준으로 각 예측을 분류한다. 예측 상자가 존재하고 $\text{IoU} \geq \theta$ 이면 올바른 탐지인 TP(True Positive), $\text{IoU} < \theta$ 이면 잘못된 탐지인 FP(False Positive), 정답 상자에 대응하는 예측이 아예 없으면 탐지 누락인 FN(False Negative)으로 분류된다. 예를 들어 $\theta = 0.5$ 로 설정했을 때, IoU가 0.7인 예측은 TP, IoU가 0.3인 예측은 FP, 탐지되지 못한 정답 상자는 FN으로 분류된다.

이를 바탕으로 정밀도(Precision)는 전체 예측 중 올바른 탐지의 비율($\text{TP} / (\text{TP} + \text{FP})$)을, 재현율(Recall)은 전체 정답 중 실제로 탐지된 비율($\text{TP} / (\text{TP} + \text{FN})$)을 나타낸다. 탐지 신뢰도 임계값을 변화시키면 정밀도와 재현율이 상충 관계로 변화하는데, 이 곡선의 아래 넓이를 단일 클래스에 대한 AP(Average Precision)라 한다. 전체 클래스에 대한 AP를 평균한 값이 mAP(mean Average Precision)로, 탐지 모델의 종합적인 성능을 비교하는 데 널리 사용되는 지표이다.

III. 관련 연구

1. 소프트웨어 정의 NDP 플랫폼 연구

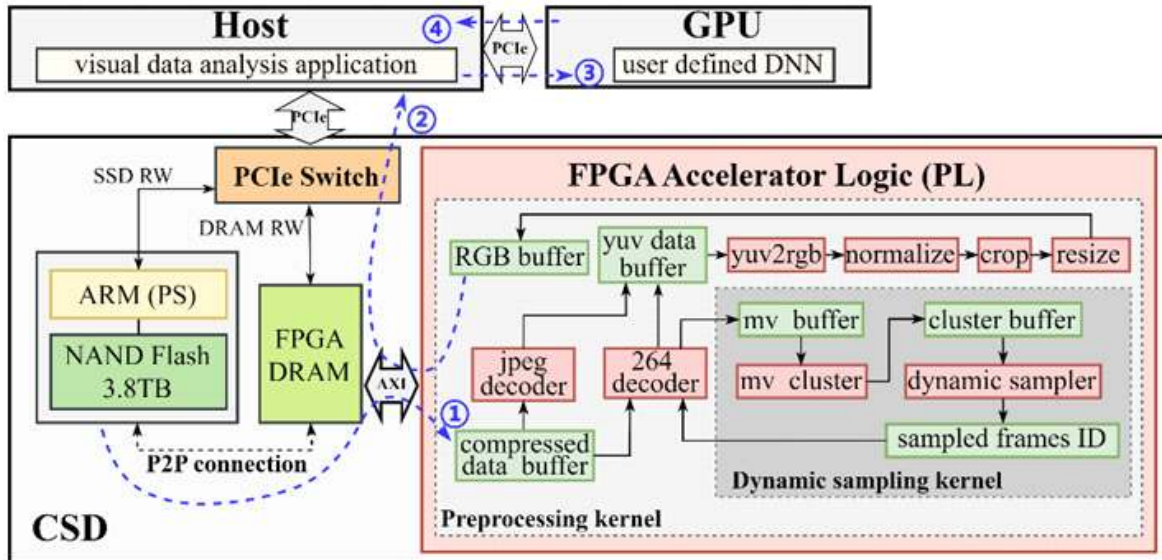


[그림 9] Near Data Processing 플랫폼 아키텍처 [4]

Yu et al.은 SPDK와 NVMe-oF를 활용하여 추가적인 하드웨어 수정 없이 소프트웨어만으로 구현 가능한 NDP 플랫폼[4]을 제안하였다. 제안된 시스템은 호스트 서버와 스토리지 서버를 ConnectX-5 NIC으로 직접 연결하는 구성을 채택하며, 스위치나 라우터와 같은 중간 네트워크 장비를 제거함으로써 성능 간섭을 최소화하였다. 연산 오프로딩 인터페이스는 NVMe-oF 프로토콜의 커스텀 opcode 메커니즘을 통해 구현하였으며, 호스트 측에서는 fiemap 시스템 콜을 통해 파일의 LBA(Logical Block Address) 범위를 조회하고 이를 연산 요청 정보와 함께 H2C PDU로 캡슐화하여 스토리지 서버로 전송한다. 스토리지 서버에서는 SPDK의 NVMe-oF Target 서브시스템을 확장하여 해당 요청을 수신하고, PDU 헤더의 data transfer bit를 조작함으로써 연산 결과를 C2H PDU로 전환하여 호스트에 반환하는 양방향 전송 메커니즘을 구현하였다. 이를 통해 대용량 파일 검색, 필터링, 집계 등 데이터 지역성이 중요한 연산을 스토리지 노드로 오프로딩할 수 있음을 보였다. 다만 해당 연구는 시스템 설계 및 구현에 초점을 맞추고 있으며, 정량적 성능 평가는 향후 연구 과제로 남겨두었다.

2. 영상 분석 파이프라인의 연산 오프로딩 연구

Zhao et al.은 Samsung SmartSSD를 CSD 플랫폼으로 활용하여 DNN 기반 영상



[그림 10] ISVDA 아키텍처 [5]

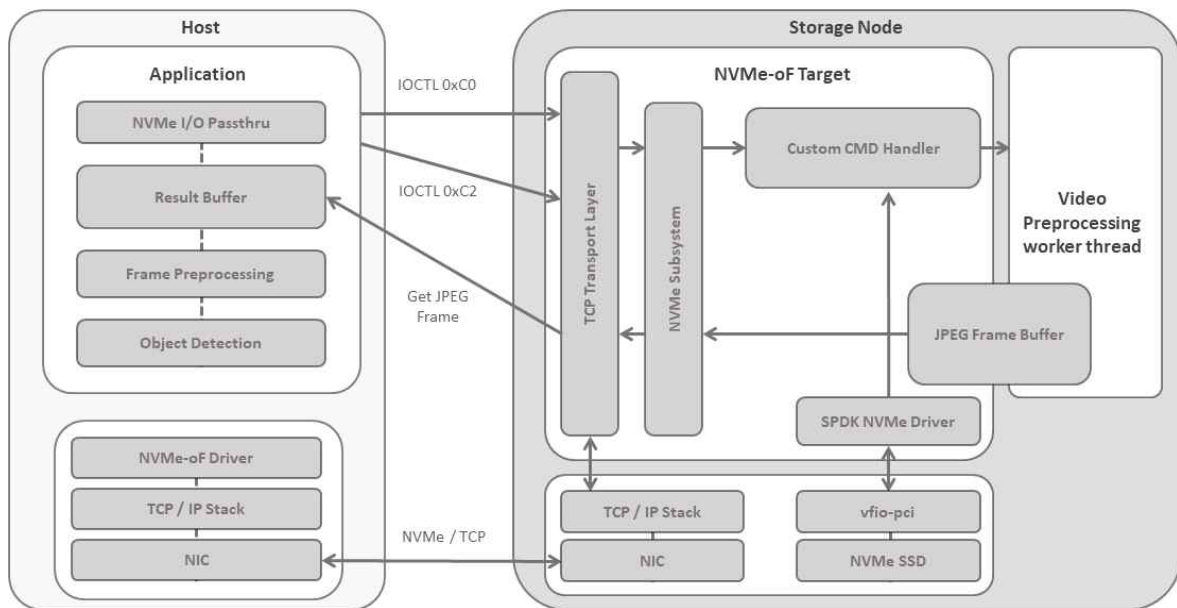
분석 워크로드의 전처리 과정을 스토리지 장치 내부로 오프로딩하는 ISVDA(In-Storage Visual Data Analysis Accelerator)[5]를 제안하였다. 기존 방식에서는 압축된 영상 데이터를 호스트 CPU로 전송하여 디코딩한 뒤 GPU로 전달하는 구조를 따르며, YOLOv8 기반 추론 실험에서 디코딩 및 리사이징·정규화 등의 전처리 단계가 전체 분석 파이프라인 시간의 상당 부분을 차지함을 확인하였다. ISVDA는 이를 해결하기 위해 디코딩, 색상 변환, 정규화, 크로핑, 리사이징 등의 전처리 작업 전체를 CSD의 FPGA 로직에서 수행하도록 설계하였으며, 단순 전처리 오프로딩에서 더 나아가 압축 비디오의 모션 벡터를 DBSCAN 클러스터링으로 분석하여 프레임 샘플링 간격을 동적으로 조정하는 기법을 함께 적용하였다. Intel i7-9700 CPU, RTX 4090 GPU 환경에서 YOLOv8을 기준으로 측정한 결과, JPEG 이미지에 대해 720p 해상도에서 6.63배, 1080p에서 6.98배의 처리 속도 향상을 달성하였으며, H.264 비디오에 대해서는 720p 기준 1.35배, 1080p 기준 1.53배의 향상을 보였다. 동적 샘플링을 포함한 종합 질의 처리 성능에서는 CPU 기반 대비 평균 2.8배의 속도 향상과 최대 79.7%의 데이터 전송량 절감 효과를 달성하였다. 그러나 해당 연구는 전용 FPGA 로직이 내장된 SmartSSD를 전제로 하므로, 범용 CPU 기반의 스토리지 노드에서 소프트웨어만으로 구현되는 환경과는 하드웨어 구성 면에서 차이가 있다.

IV. 설계 및 구현

1. 시스템 아키텍처

본 장에서는 분리형 스토리지 환경에서 영상 객체 탐지 워크로드의 전처리를 스토리지 노드로 오프로딩하는 NDP 플랫폼의 설계와 구현 세부를 기술한다. 관련 연구에서 살펴본 바와 같이 CSD 기반 NDP는 전용 하드웨어를 필요로 하여 범용 서버 환경에의 적용이 어렵고, 로컬 스토리지에 한정된다는 현실적 제약이 있다. 이에 본 연구에서는 추가적인 하드웨어 수정 없이 구현 가능한 소프트웨어 정의 NDP 방식을 채택하였다. 구체적으로 SPDK NVMe-oF Target의 커스텀 명령어 핸들러 인터페이스를 확장하여 스토리지 노드에서 영상 디코딩과 프레임 샘플링을 수행하고, 처리된 결과 프레임만을 호스트 노드로 전달하는 구조를 구현하였다.

가. 전체 시스템 구성



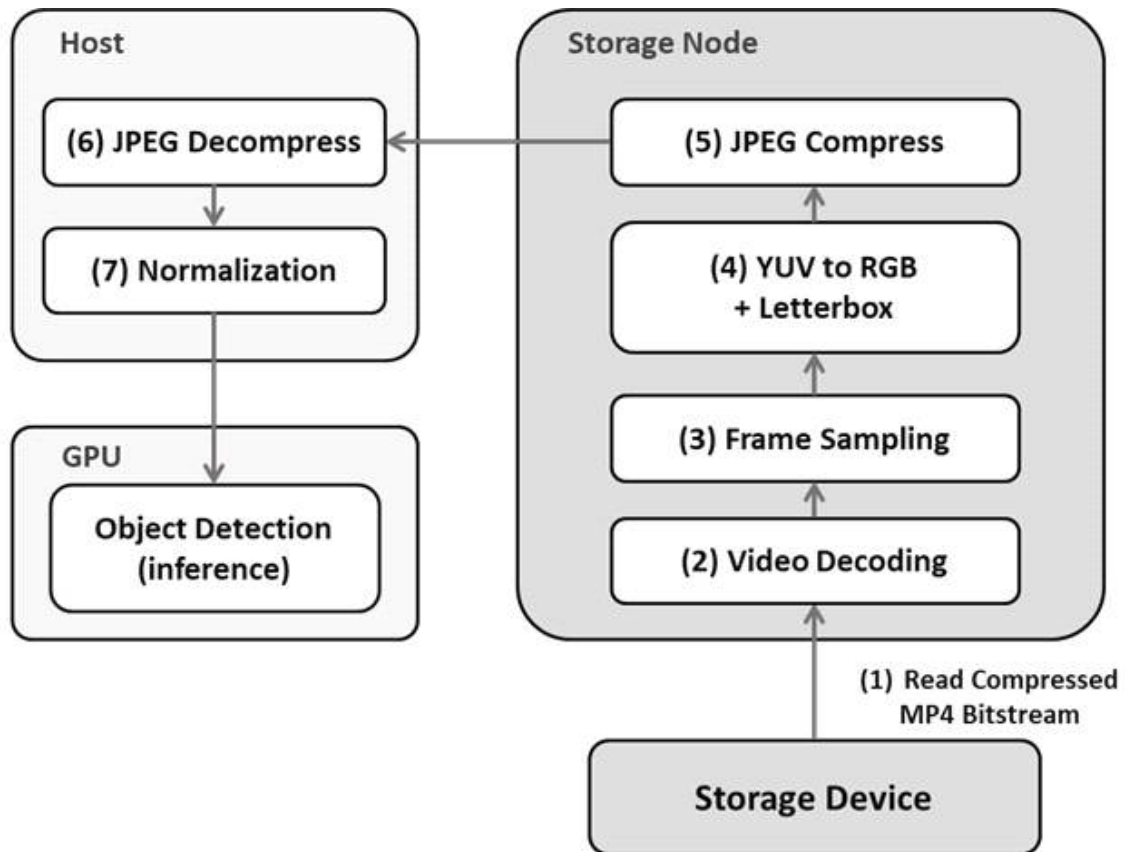
[그림 11] 제안하는 NDP 플랫폼의 시스템 아키텍처

그림 11은 제안하는 NDP 플랫폼의 전체 구성을 나타낸다. 시스템은 호스트 노드와 스토리지 노드로 구성되며, 두 노드는 NVMe/TCP 프로토콜을 통해 직접 연결된다. 스위치 등 중간 장비를 경유하지 않는 직접 연결 구성을 채택하여 네트워크 장비에 의한 추가 지연을 배제하고자 하였다.

호스트 노드의 애플리케이션은 NVMe I/O 패스스루(Passthru) 인터페이스, 결과 버퍼(Result Buffer), 프레임 전처리(Frame Preprocessing), 객체 탐지(Object Detection)의 네 요소로 구성된다. NVMe I/O 패스스루는 ioctl 기반으로 스토리지 노드에 벤더 특화 명령어를 전달하며, 결과 버퍼는 수신된 JPEG 청크를 보관한다. 이후 타겟에서 재압축한 프레임 전처리와 GPU 기반 객체 탐지가 순차적으로 수행된다. 커널 공간에는 NVMe-oF Driver, TCP/IP 스택, NIC가 위치하여 표준 NVMe/TCP 통신 경로를 담당한다.

스토리지 노드는 크게 NVMe-oF Target, 영상 전처리 워커 스레드, JPEG 프레임 버퍼로 구성된다. NVMe-oF Target 내부는 TCP Transport Layer, NVMe Subsystem, Custom CMD Handler의 계층 구조를 이루며, Custom CMD Handler는 수신된 벤더 특화 명령어의 종류에 따라 처리 흐름을 결정한다. 영상 전처리 워커는 NVMe-oF Target의 리액터와 독립된 별도의 스레드로 동작하여, 디코딩과 프레임 샘플링 등 CPU 집약적인 전처리 연산이 리액터의 I/O 처리를 방해하지 않도록 설계되었다. 처리된 프레임은 JPEG 프레임 버퍼에 보관되며, CMD Handler는 0xC2 명령 수신 시 이 버퍼에서 데이터를 읽어 호스트로 반환한다. 커널 공간에는 NVMe 장치를 사용자 공간에 노출하는 vfio-pci와 TCP/IP 스택이 위치하며, SPDK NVMe 드라이버는 사용자 공간에서 직접 장치에 접근한다. 각 노드의 하드웨어 사양 및 실험 환경 구성은 5장에서 기술한다.

전체 동작 흐름은 크게 세 단계로 구성된다. 첫째, 호스트는 처리 대상 영상 파일의 익스텐트(extent) 정보를 fiemap으로 획득하고, 이를 NDP 처리 트리거 명령어(0xC0)에 담아 스토리지 노드로 전송한다. 둘째, 스토리지 노드의 Custom CMD Handler가 명령을 수신하면 영상 전처리 워커 스레드를 실행하고, 워커는 NVMe SSD에서 영상 데이터를 읽어 전처리 파이프라인을 수행한 뒤 결과를 JPEG 프레임 버퍼에 저장한다. 셋째, 호스트는 결과 회수 명령어(0xC2)를 반복 전송하여 2MB 단위로 분할된 JPEG 프레임을 수신하며, 스토리지 노드는 전송 완료 후 해당 버퍼를 해제한다.



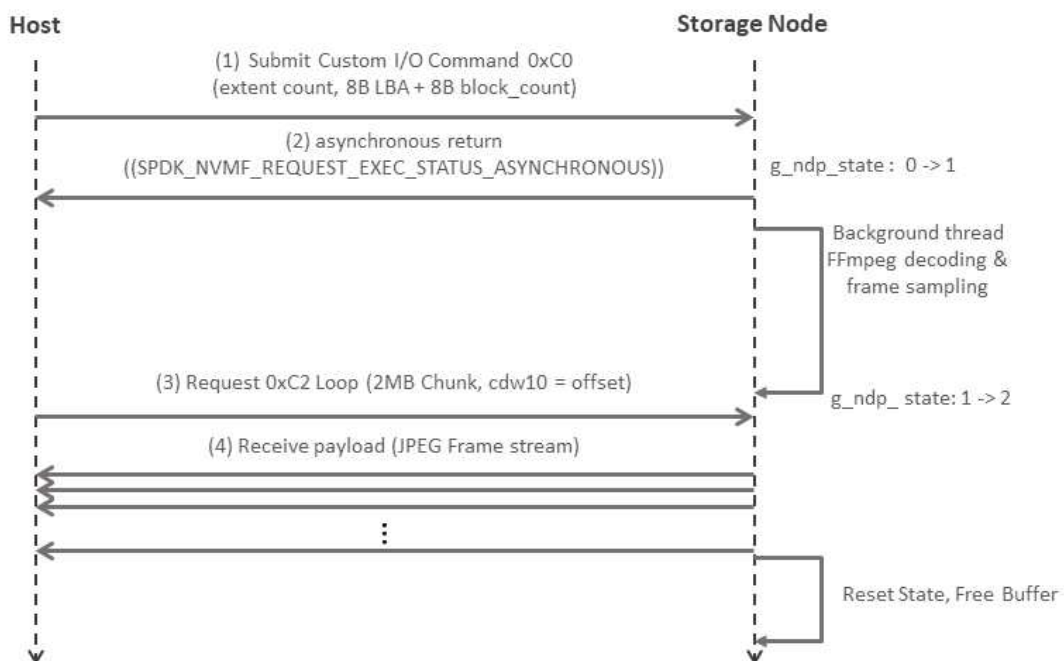
[그림 12] NDP 시스템 YOLO 영상 객체 탐지 전처리 파이프라인

그림 12는 전체 데이터 처리 관점에서의 흐름을 나타낸다. 스토리지 노드에서는 압축된 영상 비트스트림을 읽어 H.264 소프트웨어 디코딩, 프레임 샘플링, YUV→RGB 색상 변환, Resize/Letterbox 처리, JPEG 압축의 순서로 전처리가 수행된다. 호스트에서는 수신된 JPEG 프레임을 복원하고 픽셀 값을 정규화한 뒤 GPU에서 객체 탐지 추론을 수행한다. 이처럼 데이터 집약적인 전처리 연산을 스토리지 노드에서 수행하고 압축된 결과만을 전달함으로써, 호스트로 전송되는 데이터 양과 네트워크 전송 부하를 줄이는 것이 제안하는 시스템의 핵심 설계 목표이다.

2. NDP 명령어 인터페이스 설계

NVMe 명령어 집합에서 0xC0~0xFF 범위의 opcode는 NVM Express 표준이 벤더 특화 영역으로 예약한 구간이다. 본 연구에서는 이 범위에서 0xC0과 0xC2를 각각 NDP 처리 트리거와 결과 회수 명령어로 정의하고, SPDK NVMe-oF Target의 I/O 명령 처리 경로에 직접 분기를 추가하여 등록하였다. 두 명령어는 호스트가 커널 NVMe-oF 드라이버를 통해 `ioctl(nvme_fd, NVME_IOCTL_IO_CMD, &cmd)` 형태의 I/O 패스스로 전달되며, NVMe/TCP를 통해 스토리지 노드의 SPDK Target에 도달한다.

가. 0xC0 NDP 처리 트리거 명령어



[그림 13] 커스텀 NVMe-oF 명령어 기반 NDP 전처리 및 결과 회수 시퀀스 다이어그램

호스트는 처리 대상 영상 파일에 대해 `fiemap()` 시스템 콜로 익스텐트 목록을 획득하고, 각 익스텐트를 `[lba, block_count]` 형태의 `uint64_t` 쌍으로 직렬화하여 4KB 단위로 정렬된 버퍼에 담는다. `cdw10`에는 익스텐트 총 개수를 설정하며, 직렬화된 익스텐트 테이블 버퍼를 `nvme_passthru_cmd` 구조체의 `addr` 멤버 변수에 담아 H2C 방향으로 전송한다.

스토리지 노드의 핸들러는 IOVector 버퍼에서 익스텐트 테이블을 파싱하고, `spdk_dma_zmalloc()`으로 DMA 가능한 읽기 버퍼를 할당한 뒤, 익스텐트별로 `spdk_bdev_read`

d_blocks()를 호출하여 비동기 블록 읽기를 제출한다. 이 시점에 SPDK_NVMMF_REQUEST_EXEC_STATUS_ASYNCHRONOUS를 반환하여 리액터가 다른 요청을 계속 처리할 수 있도록 하며, g_ndp_state는 0(IDLE)에서 1(PROCESSING)로 전환된다.

모든 익스텐트 읽기가 완료되면 완료 카운터가 0이 되는 시점에 리액터 컨텍스트에서 pthread_create()를 호출하여 워커 스레드(ndp_preprocess_worker)를 생성하고 전체 리 연산 전체를 위임한다. 리액터 스레드에서 FFmpeg 기반 디코딩과 같은 연산 집약적 처리를 직접 수행하면 리액터가 블로킹되어 다른 I/O 처리에 지장을 주기 때문이다. 워커 스레드는 SPDK 리액터가 점유하는 코어 0과의 자원 간섭을 막기 위해 pthread_attr_setaffinity_np()를 통해 코어 2~15 범위에 핀닝되고 DETACHED 상태로 생성되며, 리액터는 즉시 복귀하여 정상적인 I/O 폴링을 재개한다.

워커 스레드가 전처리를 완료하면 spdk_thread_send_msg()를 통해 원래 SPDK 스레드에 완료 이벤트를 전달한다. SPDK 스레드 컨텍스트에서 실행되는 preprocess_done_on_spdk_thread()는 g_ndp_state를 2(READY)로 전환하고, 응답의 cdw0에 전체 버퍼 크기를 기록한 뒤 spdk_nvmf_request_complete()를 호출하여 0xC0 명령에 대한 최종 응답을 반환한다. 이 시점에 호스트의 ioctl 블로킹이 해제되고, 호스트는 cdw0 값으로 수신해야 할 전체 결과 크기를 파악한다.

나. 0xC2 결과 회수 명령어

호스트는 0xC0 응답의 cdw0에서 전체 크기를 확인한 뒤, cdw10에 현재 오프셋을 담은 0xC2 명령을 2MB 단위로 반복 전송한다. 스토리지 노드의 핸들러는 먼저 g_ndp_state == 2임을 확인하고, full_video_buffer + offset 위치에서 최대 2MB의 데이터를 IOV에 복사하여 C2H 방향으로 응답한다. 마지막 청크 전송이 완료되면 spdk_dma_free()로 버퍼를 해제하고 g_ndp_state를 0으로 초기화하여 다음 요청 대기 상태로 복원한다.

결과 버퍼는 고정 크기 헤더와 가변 길이 JPEG 데이터로 구성된다. 헤더는 프레임 총 수 N과 각 프레임의 JPEG 바이트 크기 배열을 포함하며, 이어서 N개의 JPEG 프레임 데이터가 연속 배치된다. 호스트는 헤더를 먼저 파싱하여 각 프레임의 오프셋과 크기를 계산한 뒤 개별 프레임을 복원한다.

3. 스토리지 노드 구현

스토리지 노드의 사용자 공간 내부에서 구동되는 SPDK Target 엔진을 수정 및 확장하여, I/O 처리 성능을 저해하지 않으면서 연산 집약적인 영상 디코딩 및 전처리 파이프라인을 수행하도록 구현한 세부 내역을 기술한다.

SPDK의 핵심 구동체인 리액터는 단일 스레드 기반의 논블로킹 폴링 방식으로 동작하므로, 디스크 읽기 오퍼레이션의 완료를 기다리며 블로킹되어서는 안 된다. 이를 보장하기 위해 `nvmf_bdev_ctrlr_custom_preprocess_cmd()` 핸들러 내부의 블록 읽기 과정은 완전 비동기식 구조로 구동된다. 핸들러는 호스트로부터 수신한 익스텐트 테이블을 순회하며 각 익스텐트의 블록 크기를 모두 합산한 뒤, 물리적으로 연속되고 NVMe 장치가 직접 접근할 수 있는 단일 DMA 버퍼를 확보하기 위해 합산된 전체 크기만큼 `spdk_dma_zmalloc()`을 호출하여 읽기 버퍼를 원자적으로 할당한다. 이후 파싱된 익스텐트 배열을 기반으로 루프를 돌며 `spdk_bdev_read_blocks()` 비동기 API를 연속 호출하여 스토리지 bdev 레이어에 I/O를 제출한다. 이때 개별 비동기 읽기 요청이 하드웨어 내부 사정에 의해 순서에 상관없이 완료되더라도 전체 완료 시점을 정확히 추적할 수 있도록, 커스텀 컨텍스트 구조체인 `ndp_request_ctx` 내에 완료 카운터인 `remaining_extents`를 두어 관리하며 각 bdev 읽기 요청 완료 시 카운터를 감소시키며, 모든 익스텐트 읽기가 완료되어 카운터가 0이 되는 시점에 `pthread_create()`를 호출하여 `ndp_preprocess_worker` 워커 스레드를 생성한다.

4.2.가에서 기술한 격리 구조에 따라 코어 2~15에 핀닝된 `ndp_preprocess_worker` 스레드는 파일 시스템이 배제된 Raw 블록 환경에서 동작하므로, 표준 디스크 파일 입출력 API를 사용할 수 없다. 따라서 버퍼 상에 존재하는 바이너리 데이터를 직접 스트리밍하기 위해 FFmpeg 라이브러리가 메모리 버퍼를 미디어 파일처럼 인식하고 무작위 접근할 수 있도록 핸들러를 확장하였다. 구체적으로는 불투명 객체 포인터를 추적하는 커스텀 `read_packet` 및 `seek_packet` 입출력 콜백 함수를 직접 구현하여 AVIOContext 구조체에 바인딩함으로써, 디코딩 엔진이 인메모리 바이너리 스트림을 성공적으로 파싱하고 내부 H.264 디코더 컨텍스트를 빌드하도록 유도한다. 디코더 서브시스템이 패킷을 디코딩하여 raw YUV 프레임을 생성할 때, 전체 프레임을 처리하는 대신 내부 인덱스 카운터를 기반으로 매 5번째 프레임만 추출하는 조건문을 적용하여 20% 비율의 고속 다운샘플링을 수행한다. 필터링을 통과한 프레임은 FFmpeg의 `sws_scale` 모듈을 거쳐 디바이스 독립적인 정수형 RGB24 포맷으로 일차 변환된다.

추출된 이미지를 객체 탐지 모델의 입력 텐서 규격과 일치시키기 위해 데이터 재배열 및 수학적 전처리를 수행한다. 필터링을 통과한 프레임은 FFmpeg의 sws_scale 모듈을 통해 YUV 포맷에서 416×416 크기의 RGB24 포맷으로 색공간 변환과 레터박스 리사이징을 단일 연산으로 수행한다. 원본 종횡비를 보존하면서 416×416에 맞추기 위해 남은 영역은 회색(128) 패딩으로 채운다. 변환 결과는 HWC(Height-Width-Channel) uint8 포맷으로 유지되며, libjpeg 라이브러리를 통해 압축 품질 85% 기준으로 JPEG 바이너리로 인코딩된다. CHW(Channel-Height-Width) float 변환 및 정규화는 스토리지 노드에서 수행하지 않고 호스트 측으로 위임한다. 가공이 완료된 실수형 텐서 데이터는 용량이 매우 커서 네트워크 전송에 부적합하므로, 호스트로의 송신 부하를 줄이기 위해 libjpeg 라이브러리를 사용해 압축 품질 85% 설정을 기반으로 가변 길이 JPEG 이미지 바이너리로 재압축을 수행한다. 각 프레임의 가변 압축 크기를 실시간으로 추적하기 위해 임시 힙 메모리 영역에 가변 버퍼 리스트를 유지하며 프레임별 바이트 사이즈를 배열에 기록한다.

모든 샘플링 프레임의 압축이 완료되면 최종적으로 호스트가 읽어갈 대용량 결과 공간을 할당하기 위해 독립된 워커 스레드 내부에서 spdk_dma_zmalloc()을 직접 호출하여 단일 연속 공간의 최종 결과 버퍼인 full_video_buffer를 확보한다. 이 버퍼의 맨 앞 부분에는 총 프레임 수와 가변 크기 배열로 구성된 헤더 메타데이터를 배치하고, 이어서 힙 메모리에 임시 적재되어 있던 가변 JPEG 바이너리들을 순차적으로 복사하여 단일 바이트 스트림으로 직렬화하는 작업을 완수한다. 4.2의 메시지 패싱 루틴을 거쳐 리액터 컨텍스트로 복귀하여 실행되는 preprocess_done_on_spdk_thread() 함수는 넘겨받은 최종 결과물 구조체를 전역 변수에 안전하게 바인딩하고 전역 상태를 완료 상태로 갱신한다. 마지막으로 역할이 다한 초기 비동기 디스크 읽기 버퍼를 spdk_free()로 즉각 해제하여 스토리지 노드의 메모리 자원을 환원함으로써 트리거 명령에 대한 전체 라이프사이클을 안전하게 마무리한다.

4. 호스트 노드 구현

스토리지 노드로부터 전처리가 완료된 영상 프레임 스트림을 수신하고, 이를 호스트의 메모리 공간 내에서 파싱 및 복원하여 GPU 가속 기반의 Darknet 객체 탐지 엔진과 호스트 애플리케이션의 구현 세부를 기술한다.

호스트 애플리케이션은 0xC0 트리거 명령 전송에 앞서, FS_IOC_FIEMAP ioctl을 통

해 대상 MP4 파일의 물리적 저장 위치를 조회한다. 구체적으로 fiemap 구조체를 힙에 할당한 뒤 파일 디스크립터에 FS_IOC_FIEMAP을 호출하여 파일을 구성하는 각 익스텐트의 물리적 바이트 오프셋(fe_physical)과 길이(fe_length)를 획득하고, 이를 NVMe 블록 크기(4096B)로 나누어 [lba, block_count] 쌍의 배열로 변환한다. 변환된 익스텐트 배열은 [lba0, blk0, lba1, blk1, ...] 순서로 4096B 정렬된 연속 버퍼에 직렬화되어 0xC0 명령의 데이터 페이로드로 구성되며, 익스텐트 개수는 cdw10 필드에 기입하여 스토리지 노드가 파싱 범위를 정확히 인식하도록 한다.

호스트 애플리케이션은 4.2장에서 서술한 NDP 트리거 명령어의 응답 레지스터인 cdw0를 통해 스토리지 노드가 보유한 최종 결과 직렬화 버퍼의 전체 바이트 크기를 사전에 획득한다. 이후 호스트는 해당 크기만큼 사용자 공간에 단일 연속 메모리 수신 버퍼를 할당하고, NVMe 장치의 최대 데이터 전송 크기 제약인 mdts_limit(maximum data transfer size)을 초과하지 않도록 2MB 단위로 분할하여 0xC2 결과 회수 명령어를 반복적으로 스토리지 노드에 전송한다. 이 과정은 호스트가 커널의 NVMe 패스스루 인터페이스를 경유하는 ioctl 시스템 콜을 통해 수행되며, 매 요청마다 누적된 오프셋 값을 명령어 구조체의 cdw10 필드에 기입함으로써 타겟 시스템이 직렬화 버퍼 내에서 정확한 전송 시작 위치를 참조하도록 제어한다. 수신 버퍼의 주소 포인터는 각 루프마다 전송된 데이터 길이만큼 순차적으로 증가하도록 포인터 연산을 적용하여, 네트워크 레이어에서의 중간 복사 오버헤드 없이 단일 수신 버퍼 상에 전처리 완료 직렬화 스트림 전체가 안전하게 병합되도록 구현하였다.

수신이 완료된 통합 결과 버퍼는 고정 크기의 헤더 메타데이터 영역과 가변 길이의 JPEG 바이너리 페이로드 영역으로 구분되어 있으므로, 호스트 애플리케이션은 이를 개별 프레임 단위로 분할하기 위한 인메모리 파싱 루틴을 수행한다. 버퍼 최전방의 헤더를 파싱하여 총 샘플링 프레임 수와 각 프레임의 가변 바이트 크기 배열을 획득하고, 이를 기반으로 페이로드 오프셋을 점프하며 개별 JPEG 데이터를 하나씩 추출한다.

추출된 인메모리 JPEG 바이너리는 디스크 I/O를 완전히 배제하기 위해 경량 디코딩 라이브러리인 stb_image로 메모리상에서 직접 복원된다. 복원된 HWC uint8 픽셀 배열은 Darknet의 이미지 표현 구조체에 매핑되어 CHW float 포맷으로 재배열되고, 각 픽셀 값은 [0.0, 1.0] 범위로 정규화되어 입력 텐서가 생성된다. Darknet 엔진은 빌드 타임의 Makefile 설정을 통해 호스트 노드의 NVIDIA GPU 및 CUDA 런타임을 활용하도록 구성되며, 생성된 입력 텐서는 network_predict 함수로 순차 전달되어 GPU 추론이 수행된다. 이로써 호스트는 디코딩 및 전처리 부담을 스토리지 노드로 오프로딩하고, 수

신된 전처리 완료 프레임에 대한 GPU 기반 객체 탐지 추론에 집중할 수 있다.

V. 실험 결과 및 분석

1. 실험 환경

가. 하드웨어 구성

본 실험은 호스트 노드와 스토리지 노드로 구성된 2노드 환경에서 수행되었다. 두 노드의 세부 사양은 표 1과 같다.

	호스트 노드	스토리지 노드
OS	Ubuntu 24.04.2 LTS	Ubuntu 22.04.5 LTS
커널	6.17.0-19-generic	6.8.0-87-generic
CPU	AMD EPYC 9554 (128 threads) @ 3.1GHz	Intel Xeon E5-2620 v4 (32 threads) @ 3.0GHz
RAM	755GB	64GB
GPU	NVIDIA RTX A6000	NVIDIA GeForce GTX 1050
네트워크	Intel X540-AT2 10GbE	Intel X540-AT2 10GbE
스토리지	-	Intel SSD P4500 2.5" U.2 NVMe 1TB

[표1] 실험 환경 구성

나. 소프트웨어 스택

각 노드에 구성된 소프트웨어 환경은 표 2와 같다.

	소프트웨어	버전
스토리지 노드	SPDK	24.09
	FFmpeg	4.4.2
	libavcodec / libswscale	58.134.100 / 5.9.100
	libjpeg-turbo	2.1.2
호스트 노드	CUDA Toolkit	12.0
	NVIDIA Driver	580.126.09
	cuDNN	9.20.0
	Darknet	hank-ai/darknet 기반
	OpenCV	4.6.0

[표2] 소프트웨어 환경 구성

스토리지 노드의 전처리 파이프라인은 FFmpeg의 libavcodec, libavformat, libswscale 서브시스템을 통해 H.264 디코딩 및 YUV→RGB 변환을 수행하고, libjpeg를 통해 품질 85% 기준으로 프레임을 재압축한다. 호스트 노드는 CUDA 12.0 및 cuDNN 9.20.0 환경에서 Darknet을 빌드하여 GPU 가속 추론을 수행하며, 객체 탐지 모델로는 YOLOv7-tiny(입력 해상도 416×416, 레터박스 적용)를 사용한다.

다. 실험 데이터셋

실험에 사용된 영상은 YouTube에서 수집한 도심 보행자 영상으로, 다수의 보행자가 등장하는 시내 환경을 촬영한 콘텐츠이다. 원본 영상은 VP9 코덱, 4K 해상도, 60fps, 약 3시간 분량이며, 스토리지 노드의 H.264 디코딩 파이프라인 호환성 확보를 위해 H.264 코덱으로 재인코딩하여 사용하였다.

처리 시간의 영상 길이 의존성을 분석하기 위해 원본 영상 앞부분을 기준으로 4종의 입력 데이터를 구성하였으며, 세부 사양은 표 3과 같다.

20% 다운샘플링 정책(매 5번째 프레임 추출) 적용 시, 60fps 기준 입력 데이터 크기별 추론 대상 프레임 수는 각각 약 180, 360, 720, 7,200프레임에 해당한다.

입력 데이터	재생 시간	파일 크기
testc_15s	15초	45.2MB
testc_30s	30초	95.1MB
testc_1min	1분	216.1MB
testc_10min	10분	2.12GB

[표3] 입력 데이터 크기

2. 비교 기준선 정의

가. 기존 호스트 중심 처리 파이프라인

기존 방식은 호스트가 NVMe-oF TCP로 마운트된 스토리지 장치(/dev/nvme1n1, ext4 파일시스템)에서 영상 파일을 직접 읽어 전처리 및 추론을 수행하는 구조이다. 호스트는 OpenCV 기반의 FFmpeg 백엔드를 통해 영상 스트림을 프레임 단위로 디코딩하고, get_image_from_stream_letterbox() 함수를 통해 416×416 레터박스 리사이징을 수행한 뒤 YOLOv7-tiny 모델에 순차적으로 투입한다. 이 방식은 영상의 모든 프레임을 호스트가 직접 처리하므로 대용량 고해상도 영상일수록 호스트의 CPU 및 메모리 자원 소모가 선형적으로 증가한다.

나. NDP 오프로딩 파이프라인

제안하는 NDP 방식은 호스트가 FS_IOC_FIEMAP ioctl을 통해 획득한 파일의 물리적 LBA 정보를 0xC0 커스텀 NVMe 명령으로 스토리지 노드에 전송하여 전처리를 위임하는 구조이다. 스토리지 노드는 NVMe 장치에서 직접 영상 블록을 읽어 H.264 디코딩, 20% 프레임 샘플링, YUV→RGB 변환, 레터박스 리사이징, JPEG 재압축을 수행하고, 결과를 0xC2 명령으로 호스트에 전달한다. 호스트는 수신된 JPEG 프레임을 복원하여 CHW float32로 변환·정규화한 뒤 YOLOv7-tiny GPU 추론 파이프라인에 투입한다.

본 실험에서는 NDP 파이프라인을 두 가지 조건으로 구성한다. NDP (20% 샘플링) 조건은 H.264 디코딩, 20% 프레임 샘플링, YUV→RGB 변환, 레터박스 리사이징, JPEG 재압축을 수행하는 기존 제안 방식이다. NDP (샘플링 없음) 조건은 샘플링을 제외한 전 프레임에 대해 동일한 전처리를 수행하며, 샘플링의 기여도를 독립적으로 분리하기 위한

비교 기준으로 설정하였다.

다. 측정 방법 및 지표 정의

전처리 시간은 입력 영상이 추론 가능한 상태로 변환되기까지의 시간으로 정의한다.

측정 항목	Baseline	NDP
전처리 시간	호스트 디코딩 + 리사이징 시간	스토리지 노드 전처리 완료 시간 (0xC0)
전송 시간	NVMe-oF 블록 I/O 누적 시간	0xC2 결과 회수 시간
추론 시간	network_predict() 전체 누적 시간	network_predict() 전체 누적 시간
종단 간 시간	전체 파이프라인 완료까지	전체 파이프라인 완료까지
호스트 메모리	최대 상주 메모리 (RSS)	최대 상주 메모리 (RSS)
스토리지 노드 메모리	-	처리 중 RSS 증가량
데이터 전송량	NVMe-oF 블록 I/O	JPEG 직렬화 버퍼 크기

[표4] 성능 비교를 위한 측정 항목 정의

Baseline에서는 OpenCV 디코딩 및 letterbox 처리의 누적 시간을, NDP에서는 0xC0 명령 전송 후 스토리지 노드 전처리 완료까지의 대기 시간을 측정한다.

전송 시간은 영상 데이터가 스토리지에서 호스트로 이동하는 데 소요되는 시간이다. Baseline에서는 /sys/block/nvme1n1/stat의 read_ticks를 실험 전후로 기록하여 산출하며, NDP에서는 0xC2 명령으로 JPEG 버퍼를 수신하는 시간을 측정한다.

호스트 메모리는 /usr/bin/time -v의 Maximum resident set size를 기준으로 하며, darknet 프로세스를 직접 감싸는 형태로 실행하여 프로세스 자체의 RSS를 측정한다.

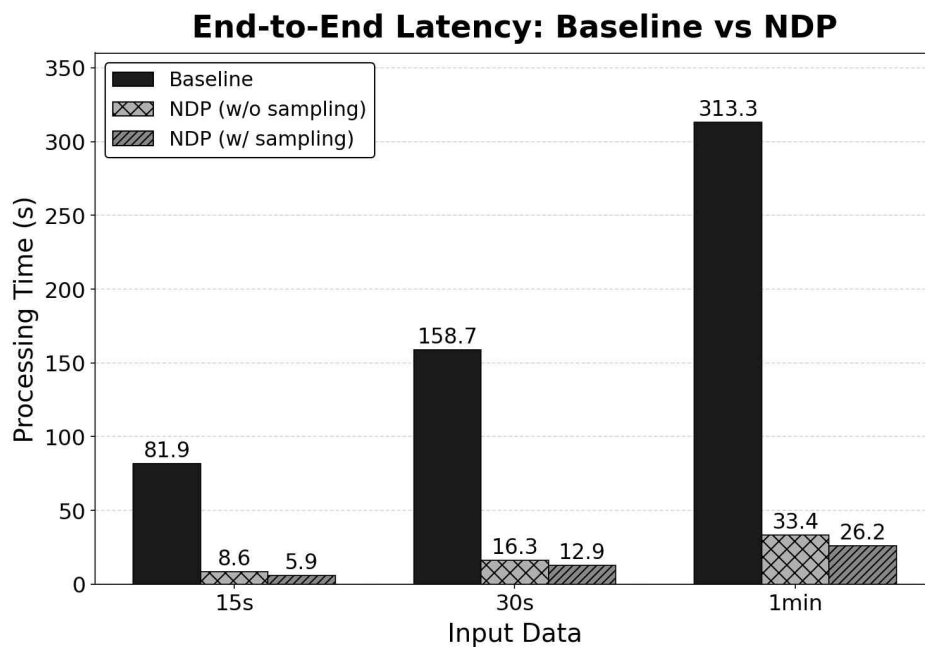
스토리지 노드 메모리는 /proc/[nvmf_tgt_pid]/status의 VmRSS를 0.3초 간격으로 샘플링하여 전처리 연산 전후 증가분을 측정한다.

프레임 샘플링은 스토리지 노드에서 수행되며, 이는 데이터 전송량 및 호스트 추론 부하를 동시에 감소시키는 NDP 오프로딩의 설계상 기능이다.

샘플링 레이트는 1/5(매 5번째 프레임)으로 설정하였다. 입력 영상이 60fps 기준이므로 샘플링 후 유효 프레임 레이트는 12fps에 해당하며, 이는 도심 보행자 검출과 같이 객체의 이동 속도가 상대적으로 느린 응용에서 연속 프레임 간 중복 정보를 제거하면서도 검출 연속성을 유지하기에 충분한 수준이다.

3. 파이프라인 처리 지연 분석

가. 종단 간 지연 시간



[그림 14] 종단 간 지연 시간

그림 14는 각 입력 데이터에 대한 Baseline, NDP (w/ sampling), NDP (w/o sampling)의 종단 간 지연 시간을 비교한 결과이다.

NDP (w/ sampling)는 모든 입력 데이터에서 Baseline 대비 약 11배의 지연 시간 단축을 달성하였다. 속도 향상 비율이 11.1x~11.6x 범위로 안정적으로 유지되는 것은 두 파이프라인 모두 지연 시간이 프레임 수에 선형 비례하기 때문이다. NDP (w/o sampling)는 샘플링을 제외한 전 프레임을 처리하므로 NDP (w/ sampling) 대비 약

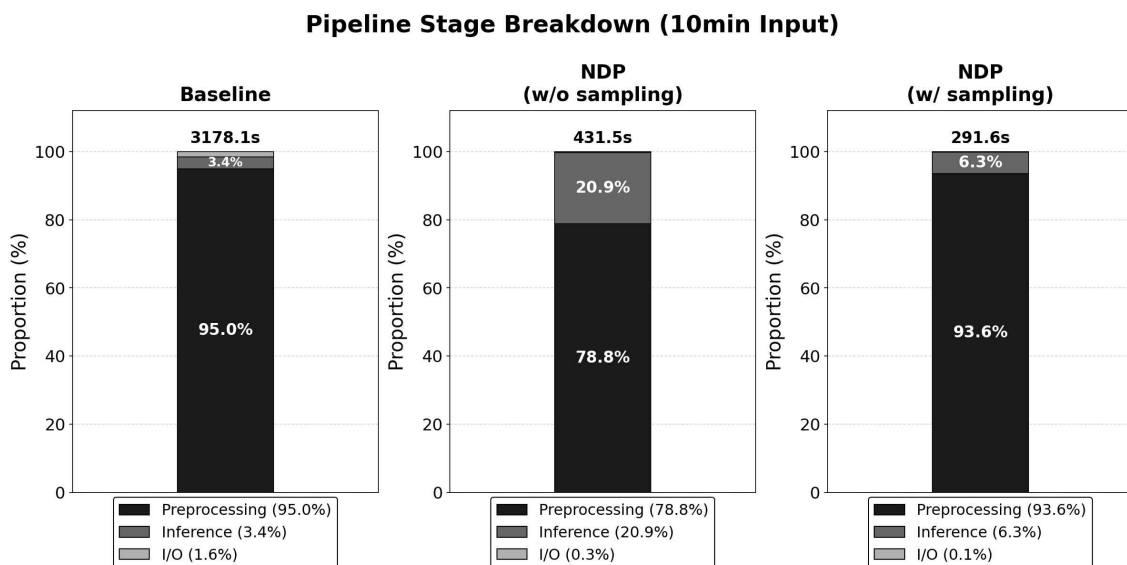
3~4배 지연이 증가하나, Baseline 대비 약 9~10배의 단축을 달성하였다. 이는 샘플링을 적용하지 않은 경우에도 전처리 오프로딩 자체의 효과가 유효함을 보인다.

Baseline에서는 종단 간 시간의 97% 이상이 전처리에 소요되며(testc_15s 기준: 82.4s 중 79.3s), 이는 호스트가 4K raw 프레임 전체를 직접 디코딩·변환하는 구조에 기인한다. NDP는 전처리를 스토리지 노드로 오프로딩함으로써 호스트 연산 부하를 제거하고, 20% 샘플링을 추가 적용함으로써 추론 대상 프레임 수를 줄여 추가적인 지연 단축 효과를 달성한다. 각 단계별 세부 시간 분해는 5.3.나에서 분석한다.

NDP는 모든 입력 데이터에서 약 11배의 지연 시간 단축을 달성하였다. 속도 향상 비율이 11.1x~11.6x 범위로 안정적으로 유지되는 것은 두 파이프라인 모두 지연 시간이 프레임 수에 선형 비례하기 때문이다.

Baseline에서는 종단 간 시간의 97% 이상이 전처리에 소요되며(testc_15s 기준: 82.4s 중 79.3s), 이는 호스트가 4K raw 프레임 전체를 직접 디코딩·변환하는 구조에 기인한다. NDP는 전처리 및 샘플링을 스토리지 노드로 오프로드하고 샘플링된 프레임만 호스트로 전달함으로써 호스트 지연 시간을 구조적으로 단축한다. 각 단계별 세부 시간 분해는 5.3.나에서 분석한다.

나. 지연 시간 세부 분석



[그림 15] 파이프라인 단계별 지연 시간 분해

각 파이프라인의 지연 시간 구성을 단계별로 분석하여 병목 구간을 보이고자 한다.

그림 15는 10min 입력 데이터에 대한 Baseline과 NDP의 파이프라인 구간별 지연 시간 비율을 나타낸다.

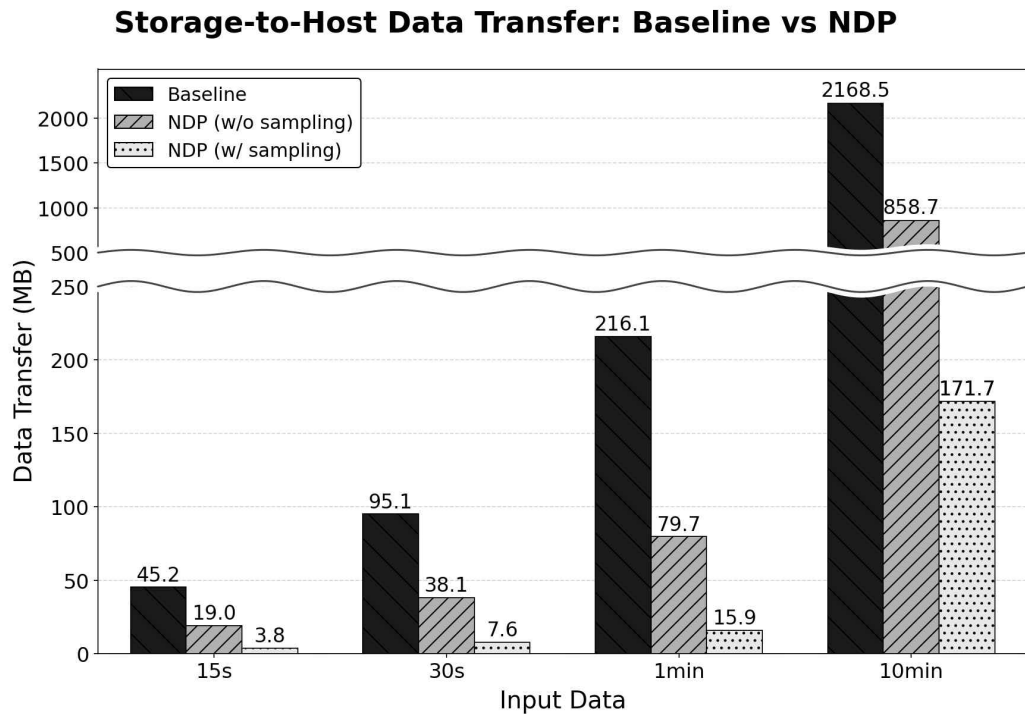
Baseline의 전체 지연 시간은 3178.1s이며, 이 중 전처리(Preprocessing)가 95.0% (약 3019.8s)를 차지하는 압도적 병목임을 확인하였다. 추론(Inference)은 3.4%, I/O는 1.6%에 불과하여 추론 연산 자체는 성능 병목이 아님을 확인하였다. Baseline에서 전처리 병목의 핵심은 `mat_to_image` 연산이다. OpenCV VideoCapture를 통해 획득한 4K 프레임(3840×2160×3)을 Darknet이 요구하는 float32 CHW 포맷으로 변환하는 이 연산이 전체 전처리 시간의 대부분을 차지한다.

NDP(w/ sampling)는 전체 지연 시간 291.6s로 Baseline 대비 약 10.9배 단축되었다. 전처리 비율은 93.6%로 여전히 지배적이나, 절대 소요 시간은 약 273.0s로 Baseline(약 3019.8s) 대비 약 11.1배 감소하였다. 이는 NDP가 스토리지 노드에서 FFmpeg를 통해 디코딩과 전처리를 직접 수행하고 416×416 JPEG로 압축하여 전달함으로써, 호스트에서 `mat_to_image` 연산이 구조적으로 제거되기 때문이다. 호스트는 416×416 JPEG (약 0.024MB)를 복원하여 2.077MB float32 CHW로 변환하는 작업만 수행하므로, 변환 대상이 Baseline 대비 약 48배 축소된다.

NDP(w/o sampling)는 291.6s보다 높은 431.5s를 기록하였다. 샘플링을 적용하지 않아 처리 프레임 수가 증가함에 따라 Inference 비율이 20.9%로 상승하였으며, 이는 NDP(w/ sampling) 조건(6.3%) 대비 약 3.3배 높은 수치이다. 전송(I/O) 비율은 0.3%로 NVMe/TCP 전송 오버헤드가 전체 성능에 실질적 영향을 미치지 않음을 확인하였다.

4. 데이터 전송량 및 전송 시간 비교

그림 16은 각 입력 데이터에 대한 Baseline, NDP (w/ sampling), NDP (w/o sampling)의 스토리지-호스트 간 데이터 전송량을 비교한 결과이다. Baseline의 데이터 전송량은 측정된 블록 I/O량이 영상 파일 크기와 거의 일치하며, 이는 호스트가 NVMe-oF를 통해 파일 전체를 블록 단위로 읽기 때문이다. NDP (w/ sampling)는 1/5 샘플링과 416×416 JPEG 압축의 결합 효과로 데이터 전송량이 92~93% 감소하며, 이 비율은 입력 데이터 길이와 무관하게 일정하게 유지된다. NDP (w/o sampling)는 JPEG 압축만 적용되어 Baseline 대비 58~60%의 전송량 감소를 달성하였다. 이는 샘플링이 전송량 감소에 기여하는 비중이 압축보다 압도적임을 보여준다.

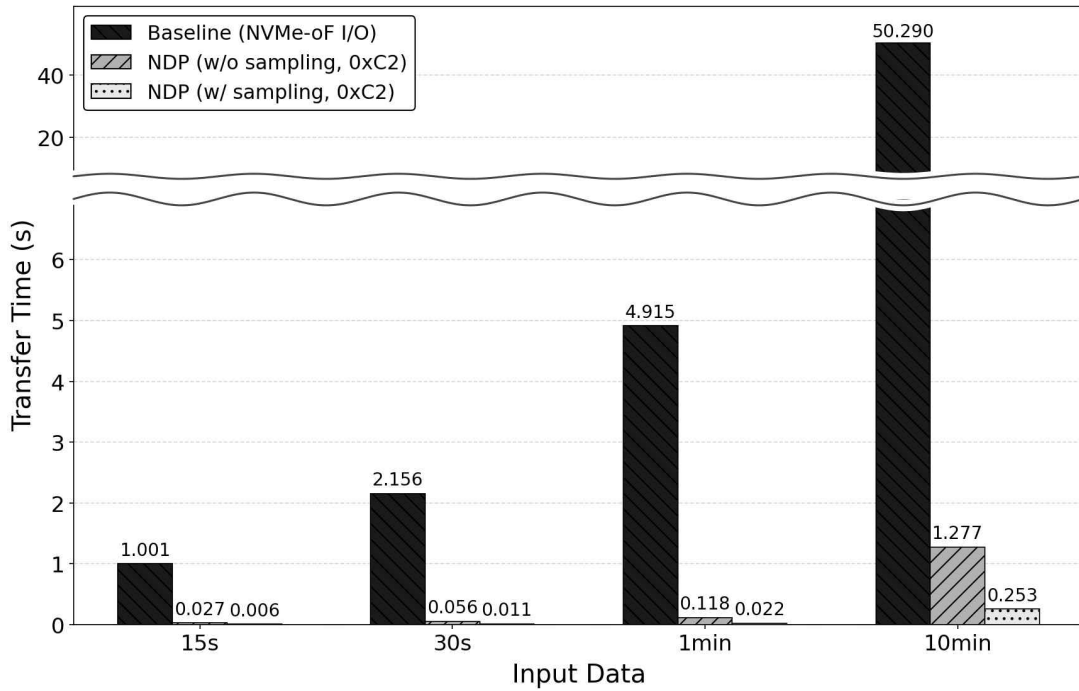


[그림 16] 스토리지-호스트 간 데이터 전송량 비교

그림 17은 전송 시간을 비교한 결과이다. Baseline의 NVMe-oF 블록 I/O 시간은 15s 기준 1.001s에서 10min 기준 50.290s로 입력 길이에 선형 비례하여 증가한다. NDP (w/ sampling)의 0xC2 전송 시간은 0.006~0.253s, NDP (w/o sampling)는 0.027~1.277s로, 두 조건 모두 Baseline 대비 전송 시간이 현저히 낮다.

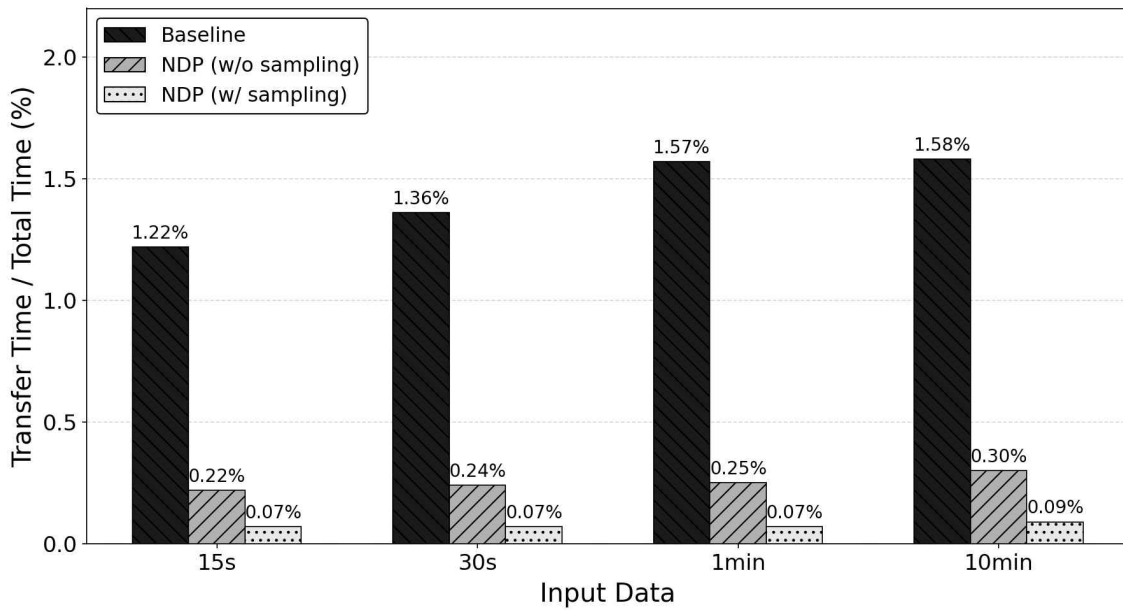
그림 18은 전체 실행 시간 대비 데이터 전송 시간의 비율을 비교한 결과이다. Baseline은 1.22~1.58%, NDP (w/ sampling)는 0.07~0.09%, NDP (w/o sampling)는 0.22~0.30%로, 세 조건 모두 전송 시간이 전체 파이프라인에 미치는 영향은 무시 가능한 수준이다. 이는 10GbE 직접 연결 환경에서 NVMe/TCP 전송 오버헤드가 실질적인 병목으로 작용하지 않음을 확인한 결과이다.

Storage-to-Host Transfer Time: Baseline vs NDP



[그림 17] 스토리지-호스트 간 데이터 전송시간 비교

Data Transfer Time as % of Total Latency



[그림 18] 스토리지-호스트 간 전체 실행시간 대비 데이터 전송시간 비율 비교

5. 메모리 사용량 분석

표 5는 각 입력 데이터에 대한 호스트 및 스토리지 노드의 메모리 사용량을 비교한 결과이다.

입력 데이터	Baseline 호스트	NDP 호스트	NDP 스토리지 증가분	NDP 시스템 합계	메모리 사용량 감소율
testc_15s	5,748 MB	533 MB	131 MB	664 MB	88.4%
testc_30s	5,799 MB	537 MB	156 MB	693 MB	88.1%
testc_1min	5,799 MB	544 MB	154 MB	698 MB	87.0%
testc_10min	5,839 MB	693 MB	392 MB	1,085 MB	81.4%

[표5] 호스트 및 시스템 전체 메모리 사용량 비교

Baseline의 호스트 RSS는 입력 데이터 길이와 무관하게 ~5,800 MB를 유지한다. 이는 5.3.나에서 분석한 바와 같이 4K raw 프레임을 매 프레임 float32로 변환하여 호스트 메모리에 적재하는 구조로 분석된다. NDP 호스트 RSS의 고정 구성분인 ~530 MB는 YOLOv7-tiny 모델 weights 및 GPU context에 해당하며, testc_10min에서 693 MB로 증가하는 것은 0xC2 명령으로 수신한 JPEG 버퍼(163 MB)가 호스트 메모리에 일괄 적재되기 때문으로 5.4절의 데이터 전송량과 일치한다.

스토리지 노드의 base RSS(~590 MB)는 SPDK hugepage 기반 DMA 사전 할당 비용으로 NDP 유무와 무관하게 상시 점유되므로, NDP 오버헤드는 증가분만으로 평가한다. 증가분은 FFmpeg 디코딩 컨텍스트 및 JPEG 프레임 버퍼에 해당하며 처리 완료 후 즉시 해제된다.

호스트 메모리 절감분의 일부가 스토리지 노드로 이전되지만, 시스템 합계 기준으로 81~88%의 메모리 절감이 달성된다. 호스트와 스토리지 노드는 물리적으로 분리된 독립 장치이므로 호스트 메모리 절감은 호스트 워크로드 효율화에 직접 기여한다.

VI. 결론 및 향후 연구

1. 결론

본 논문에서는 분리형 스토리지 환경에서 대용량 동영상 객체 탐지 파이프라인이 야기하는 호스트 I/O 대역폭 병목 및 메모리 과부하 문제를 해결하기 위해 NVMe-oF 기반 Near-Data Processing 아키텍처를 설계하고 구현하였다. 제안 시스템은 SPDK NVMe-oF 타겟 엔진을 확장하여 커스텀 NVMe 명령(0xC0)을 수신하면 H.264 디코딩, 20% 프레임 샘플링, 레터박스 리사이징, JPEG 재압축의 전처리 파이프라인을 스토리지 노드 내부에서 직접 수행하고, 전처리 완료된 프레임만 호스트로 전달한다. 또한 연산 집약적 전처리를 격리된 워커 스레드로 오프로딩하여 SPDK 리액터의 I/O 처리 성능을 저해하지 않도록 설계하였다.

기존 호스트 중심 처리 방식을 Baseline으로 설정하여 정량적으로 평가한 결과, 제안 시스템은 모든 입력 데이터에서 약 11배의 종단 간 지연 시간 단축을 달성하였다. Baseline 전처리 지연의 최대 병목은 4K raw 프레임의 float32 변환 및 레이아웃 전치(mat_to_image, 전체의 53%)였으며, NDP는 이를 스토리지 노드로 오프로딩하여 구조적으로 제거하였다. 데이터 전송량은 프레임 샘플링과 JPEG 압축의 결합 효과로 92~93% 감소하였고, 호스트 메모리 사용량은 88~91% 절감되었다. 스토리지 노드의 메모리 증가분을 포함한 시스템 합계 기준으로도 81~88%의 메모리 절감이 달성되어, 제안하는 소프트웨어 정의 NDP가 분리형 스토리지 환경에서 데이터 전송 병목 완화와 호스트 자원 효율화를 동시에 달성하는 실질적인 최적화 수단으로 기능함을 확인하였다.

2. 향후 연구

본 구현에서 식별된 한계점을 기반으로 다음과 같은 연구 방향을 제시한다.

가. CXL 기반 메모리 시맨틱 NDP 아키텍처로의 확장

본 연구는 NVMe-oF TCP 프로토콜을 기반으로 스토리지 노드와 호스트 간 명령 및 데이터 전송을 수행하였다. 그러나 실험에 사용된 호스트 노드(Super Server)는 CXL(Compute Express Link) 인터페이스를 지원하는 플랫폼으로, 향후 NVMe-oF 기반 블록 전송 방식을 CXL 메모리 시맨틱 접근 방식으로 대체하는 연구가 가능하다.

CXL.mem 프로토콜을 활용하면 스토리지 노드가 생성한 전처리 결과 버퍼를 호스트의 가상 주소 공간에 직접 노출할 수 있어, 현재 구현에서 필요한 0xC2 명령 기반의 분할 전송 및 수신 버퍼 복사 오버헤드를 원천적으로 제거할 수 있다. 이는 대용량 영상 처리 시 전송 단계가 전체 파이프라인에서 차지하는 비중을 크게 줄일 수 있는 유망한 연구 방향이다.

나. 다중 스토리지 노드 분산 처리 아키텍처

현재 구현은 단일 스토리지 노드와 단일 호스트 간의 1:1 구조로 동작하며, 전역 상태 변수 `g_ndp_state`에 의해 한 번에 하나의 전처리 요청만 처리 가능한 직렬 구조를 갖는다. 실제 스마트 시티 및 보안 인프라 환경에서는 다수의 CCTV 스트림을 동시에 처리해야 하므로, 복수의 스토리지 노드가 영상 스트림을 분산하여 병렬 전처리하는 아키텍처로의 확장이 필요하다. 이를 위해서는 요청별 독립 컨텍스트 관리, 노드 간 부하 분산 정책, 그리고 NVMe-oF 패브릭 기반의 다중 타겟 연결 관리 메커니즘이 함께 설계되어야 한다. 나아가 스위치 기반 NVMe-oF 패브릭 토폴로지를 구성하여 호스트 한 대가 복수의 스토리지 노드에 전처리 작업을 분산 제출하고 결과를 병합하는 구조를 구현한다면, 처리 가능한 동시 스트림 수를 수평적으로 확장할 수 있다.

다. 전용 연산 가속기 탑재를 통한 전처리 성능 향상

본 연구에서 스토리지 노드의 전처리 연산은 Intel Xeon E5-2620 v4 범용 CPU에서 수행되었다. 이는 FFmpeg 디코딩, 색공간 변환, 레터박스 리사이징, JPEG 재압축 등 연산 집약적 작업을 범용 코어가 전담하는 구조로, 스토리지 노드의 컴퓨팅 자원이 제한적일수록 전처리 처리량이 병목이 될 수 있다. 향후 연구로는 SmartNIC 또는 DPU(Data Processing Unit)와 같은 전용 네트워크 연산 가속기를 스토리지 노드에 탑재하여 전처리 파이프라인의 일부 혹은 전체를 오프로딩하는 방향을 고려할 수 있다. 또한 FPGA 기반 하드웨어 가속기를 활용하여 H.264 디코딩 및 색공간 변환 단계를 하드웨어 파이프라인으로 구현하면, CPU 자원 소모 없이 고속 전처리가 가능해진다. 이러한 하드웨어 이기종 컴퓨팅 구조는 스토리지 노드의 I/O 처리 성능을 유지하면서 전처리 처리량을 대폭 향상시킬 수 있는 유망한 연구 과제이다.

라. 적응형 프레임 샘플링 및 전처리 정책

현재 구현은 매 5번째 프레임을 고정적으로 추출하는 정적 샘플링 정책을 채택하고 있다. 이는 구현의 단순성을 확보하는 반면, 영상 내 객체의 이동 속도, 장면 전환 빈도, 혼잡도 등 콘텐츠의 동적 특성을 전혀 반영하지 못한다는 한계가 있다. 향후 연구로는 프레임 간 차이값(inter-frame difference) 또는 히스토그램 기반 장면 전환 감지를 스토리지 노드 내에서 경량 연산으로 수행하여, 정보량이 높은 프레임만 선택적으로 추출하는 콘텐츠 적응형 샘플링 정책을 도입할 수 있다. 이는 동일한 네트워크 전송 부하 하에서 추론 정확도를 향상시키는 효과를 기대할 수 있으며, 샘플링 비율 자체를 호스트가 동적으로 제어할 수 있는 파라미터로 노출하는 인터페이스 설계도 함께 고려되어야 한다.

참고문헌

- [1] A. Acharya, M. Uysal, and J. Saltz, "Active disks: programming model, algorithms and evaluation," *ACM SIGOPS Oper. Syst. Rev.*, vol. 32, no. 5, pp. 81–91, Dec. 1998.
- [2] B. Gu et al., "Biscuit: a framework for near-data processing of big data workloads," *ACM SIGARCH Comput. Archit. News*, vol. 44, no. 3, pp. 153–165, Jun. 2016.
- [3] S. Seshadri et al., "Willow: a user-programmable SSD," in *Proc. 11th USENIX Conf. Operating Systems Design and Implementation (OSDI)*, 2014, pp. 67–80.
- [4] 유승훈, 한창수, 윤성준, 안성용, "SPDK와 NVMe-over-Fabric을 이용한 Near-Data Processing 플랫폼 구축," *한국소프트웨어종합학술대회 논문집*, pp. 1716-1718, 2024.
- [5] Z. Zhao, Z. Shen, and X. Cai, "ISVDA: An in-storage processing accelerator for visual data analysis," in *Proc. 42nd IEEE Int. Conf. Computer Design (ICCD)*, 2024, pp. 661–664.
- [6] NVM Express, "NVM Express over Fabrics," *nvmexpress.org*. [Online]. Available: https://nvmexpress.org/wp-content/uploads/NVMe_Over_Fabrics.pdf
- [7] NVM Express, "NVM Express TCP Transport Specification, Revision 1.1," *nvmexpress.org*, Aug. 2024.
- [8] C.-Y. Wang, A. Bochkovski, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475.
- [9] J. Redmon, "Darknet: Open source neural networks in C," *pjreddie.com*, 2016. [Online]. Available: <https://pjreddie.com/darknet>
- [10] Z. Gu, H. Li, A. Shayesteh, and V. Balakrishnan, "Performance characterization of NVMe-over-fabrics storage disaggregation," *ACM Trans. Storage*, vol. 14, no. 4, Art. 31, Nov. 2018.
- [11] A. Kashyap, S. Gugnani, and X. Lu, "Impact of commodity networks on stor

age disaggregation with NVMe-oF," in Lecture Notes in Computer Science, vol. 12614, Springer, 2021, pp. 41-56.

- [12] Z. Yang et al., "SPDK: A development kit to build high performance storage applications," in Proc. IEEE Int. Conf. Cloud Computing Technology and Science (CloudCom), 2017, pp. 154-161.
- [13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.
- [14] J. Kowalczyk et al., "Performance of YOLO based traffic analysis in high resolution video stream," Procedia Comput. Sci., 2025. <https://doi.org/10.1016/j.procs.2025.09.565>

Optimization of YOLO Video Preprocessing Pipeline via NVMe-oF-Based Near-Data Processing

Kitae Kim

Department of Information Convergence Engineering
The Graduate School
Pusan National University

Abstract

The rapid expansion of smart city infrastructure and security systems has driven growing interest in object detection technologies that leverage visual data from sources such as closed-circuit television (CCTV) cameras. In particular, deep neural network (DNN)-based object detection models such as YOLO have achieved significant improvements in inference performance. However, the video preprocessing stage required prior to inference necessitates the transfer of large-scale video data to the host system, resulting in excessive consumption of I/O bandwidth and host memory resources.

To address these challenges, this paper proposes a Near-Data Processing (NDP)-based architecture for video decoding and data preprocessing. The proposed approach offloads video decoding directly to the storage node, selectively samples only the frames required for DNN inference, and delivers preprocessed inputs conforming to the YOLO inference specification to the host. To realize this architecture, a video preprocessing computation offloading protocol based on NVMe over Fabrics (NVMe-oF) was implemented and integrated into an existing YOLO inference pipeline for performance evaluation.

The experimental results demonstrate that the proposed NDP-based video preprocessing pipeline reduces both the volume of data transferred to the host and host memory utilization, thereby optimizing the YOLO video preprocessing pipeline in a disaggregated storage environment.